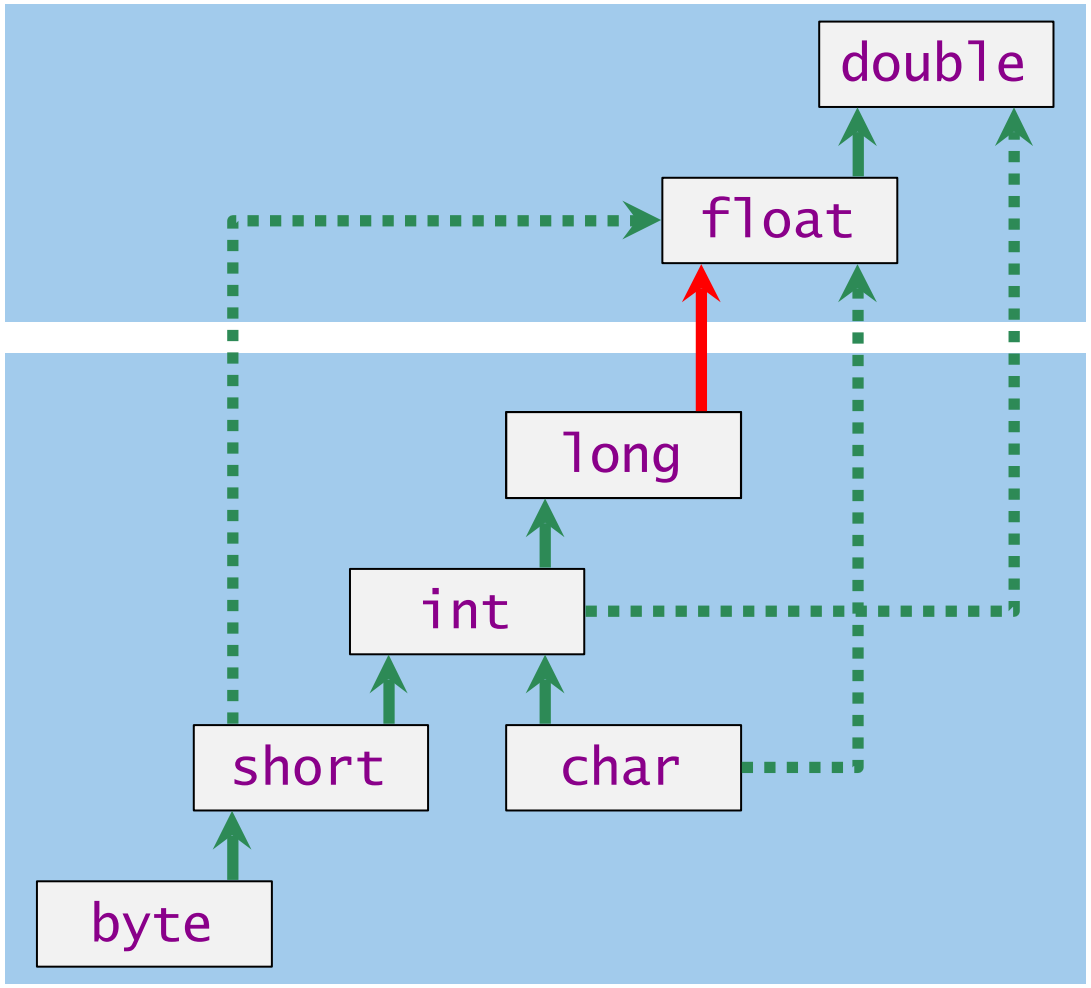
Erlaubte Implizite Typecasts – Numerische Typen



Gleitkommazahlen

ganze Zahlen, char

Konvertierung von `long` nach `double` oder von `int` nach `float` kann Information verlieren wird aber automatisch durchgeführt.

Erlaubte Implizite Typecasts – Numerische Typen

Zahlen-Klassen abgeleitet von Number

Casts zwischen diesen Klassen sind nicht erlaubt!

Beispiel *class Integer*

public int compareTo(Integer anotherInteger)

Spezifiziert durch: *compareTo* in *interface Comparable<Integer>*

the value 0 if this Integer is equal to the argument Integer;

a value less 0 if this Integer is numerically less than the argument Integer;

and a value greater than 0 if this Integer is numerically greater than the argument Integer (signed comparison).

Operatoren

Unäre Operatoren:

symbol	*name*	*types*	*L/R*	*level*
`++`	Post-inkrement	Zahl, `char`	links	2
`--`	Post-dekrement	Zahl, `char`	links	2
`++`	Pre-inkrement	Zahl, `char`	rechts	3
`--`	Pre-dekrement	Zahl, `char`	rechts	3
`+`	unäres Plus	Zahl, `char`	rechts	3
`-`	unäres Minus	Zahl, `char`	rechts	3
`!`	Negation	`boolean`	rechts	3

Operatoren

Binäre arithmetische Operatoren:

`byte`, `short`, `char` werden nach `int` konvertiert

symbol	*name*	*types*	*L/R*	*level*
*	Multiplikation	Zahl, char	links	4
/	Division	Zahl, char	links	4
%	Modulo	Zahl, char	links	4
+	Addition	Zahl, char	links	5
-	Subtraktion	Zahl, char	links	5

Konkatenation

symbol	*name*	*types*	*L/R*	*level*
+	Konkatenation	`String`	links	5

Operatoren

Vergleichsoperatoren:

symbol	*name*	*types*	*L/R*	*level*
>	größer	Zahl, char	keine	7
>=	größergleich	Zahl, char	keine	7
<	kleiner	Zahl, char	keine	7
<=	kleinergleich	Zahl, char	keine	7
==	gleich	alle	links	8
!=	ungleich	alle	links	8

Operatoren

Boolsche Operatoren:

symbol	*name*	*types*	*L/R*	*level*
&&	Und-Bedingung	boolean	links	12
||	Oder-Bedingung	boolean	links	13

Operatoren

Zuweisungsoperatoren:

symbol	*name*	*types*	*L/R*	*level*
=	Zuweisung	(links var) alle	rechts	15
*=,/=, %=, +=, -=	Zuweisung	(links var) alle	rechts	15

Für die letzte Form gilt:

$$v \circ= a \iff v = (type(v))\ (v \circ a)$$

Auswertung von Ausdrücken

Assoziativität

- Die Assoziativität entscheidet über die Reihenfolge bei Operatoren gleicher Priorität. (links = der linkeste Operator wird zuerst ausgeführt)
- Alle Operatoren einer Prioritätsgruppe haben dieselbe Assoziativität.
- Bis auf Zuweisungsoperatoren (=, +=, etc.) sind alle binären Operatoren linksassoziativ.
- unäre Operatoren, die ihr Argument rechts erwarten sind rechtsassoziativ
- unäre Operatoren, die ihr Argument links erwarten (postfix-Operatoren ++, --) sind linksassoziativ
- Der ternäre Bedingungsoperator (später) ist rechtsassoziativ

Auswertung von Ausdrücken

Assoziativität

- Die Assoziativität entscheidet über die Reihenfolge bei Operatoren gleicher Priorität. (links = der linkeste Operator wird zuerst ausgeführt)
- Alle Operatoren einer Prioritätsgruppe haben dieselbe Assoziativität.
- Bis auf Zuweisungsoperatoren (=, +=, etc.) sind alle binären Operatoren linksassoziativ.
- unäre Operatoren, die ihr Argument rechts erwarten sind rechtsassoziativ
- unäre Operatoren, die ihr Argument links erwarten (postfix-Operatoren ++, --) sind linksassoziativ
- Der ternäre Bedingungsoperator (später) ist rechtsassoziativ

Auswertung von Ausdrücken

Assoziativität

- Die Assoziativität entscheidet über die Reihenfolge bei Operatoren gleicher Priorität. (links = der linkeste Operator wird zuerst ausgeführt)
- Alle Operatoren einer Prioritätsgruppe haben dieselbe Assoziativität.
- Bis auf Zuweisungsoperatoren (=, +=, etc.) sind alle binären Operatoren linksassoziativ.
- unäre Operatoren, die ihr Argument rechts erwarten sind rechtsassoziativ
- unäre Operatoren, die ihr Argument links erwarten (postfix-Operatoren ++, --) sind linksassoziativ
- Der ternäre Bedingungsoperator (später) ist rechtsassoziativ

Auswertung von Ausdrücken

Assoziativität

- Die Assoziativität entscheidet über die Reihenfolge bei Operatoren gleicher Priorität. (links = der linkeste Operator wird zuerst ausgeführt)
- Alle Operatoren einer Prioritätsgruppe haben dieselbe Assoziativität.
- Bis auf Zuweisungsoperatoren (=, +=, etc.) sind alle binären Operatoren linksassoziativ.
- unäre Operatoren, die ihr Argument rechts erwarten sind rechtsassoziativ
- unäre Operatoren, die ihr Argument links erwarten (postfix-Operatoren ++, --) sind linksassoziativ
- Der ternäre Bedingungsoperator (später) ist rechtsassoziativ

Auswertung von Ausdrücken

Assoziativität

- Die Assoziativität entscheidet über die Reihenfolge bei Operatoren gleicher Priorität. (links = der linkeste Operator wird zuerst ausgeführt)
- Alle Operatoren einer Prioritätsgruppe haben dieselbe Assoziativität.
- Bis auf Zuweisungsoperatoren (`=`, `+=`, etc.) sind alle binären Operatoren linksassoziativ.
- unäre Operatoren, die ihr Argument rechts erwarten sind rechtsassoziativ
- unäre Operatoren, die ihr Argument links erwarten (postfix-Operatoren `++`, `--`) sind linksassoziativ
- Der ternäre Bedingungsoperator (später) ist rechtsassoziativ

Auswertung von Ausdrücken

Assoziativität

- Die Assoziativität entscheidet über die Reihenfolge bei Operatoren gleicher Priorität. (links = der linkeste Operator wird zuerst ausgeführt)
- Alle Operatoren einer Prioritätsgruppe haben dieselbe Assoziativität.
- Bis auf Zuweisungsoperatoren (=, +=, etc.) sind alle binären Operatoren linksassoziativ.
- unäre Operatoren, die ihr Argument rechts erwarten sind rechtsassoziativ
- unäre Operatoren, die ihr Argument links erwarten (postfix-Operatoren ++, --) sind linksassoziativ
- Der ternäre Bedingungsoperator ist rechtsassoziativ: a?b:c?d:e?f:g =

Auswertung von Ausdrücken

In Java werden Unterausdrücke von links nach rechts ausgewertet. D.h. um den Wert einer Operation zu berechnen:

- werte (rekursiv) alle Operanden von links nach rechts aus
- führe die Operation auf den Resultaten aus

Ausnahmen: `||`, `&&`, und der ternäre Bedingungsoperator `:?`, werten nicht alle Operanden aus (Kurzschlussauswertung).

Man sollte nie Ausdrücke formulieren, deren Ergebnis von der Auswertungsreihenfolge abhängt!!!

Auswertung von Ausdrücken

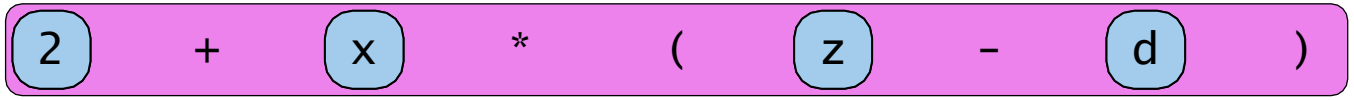
Im Folgenden betrachten wir Klammern als einen Operator der nichts tut:

symbol	*name*	*types*	*L/R*	*level*
( )	Klammerung	alle	links	0

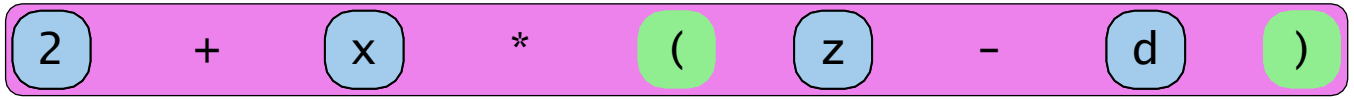
Beispiel: 2 + x * (z - d)

2	+	x	*	(	z	-	d	)

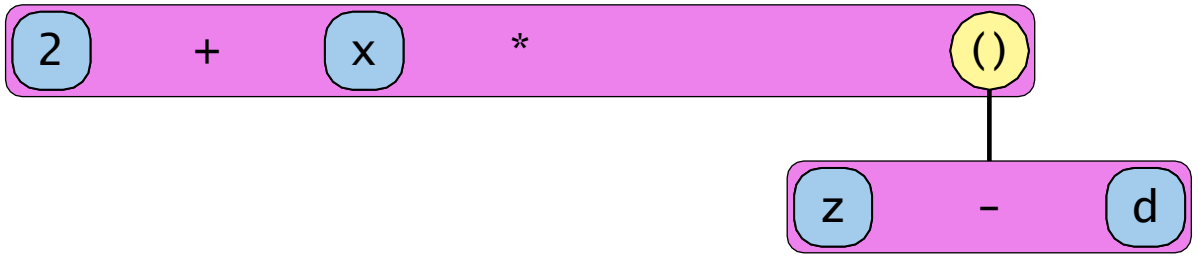
Beispiel: 2 + x * (z - d)



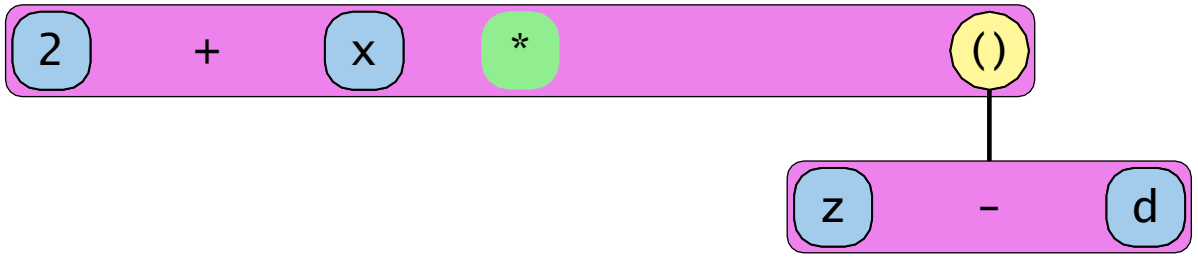
Beispiel: 2 + x * (z - d)



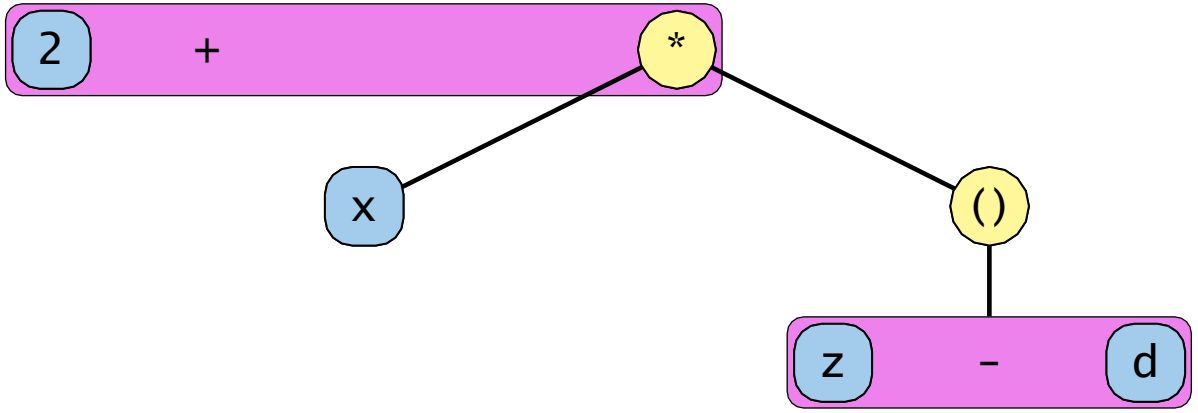
Beispiel: 2 + x * (z - d)



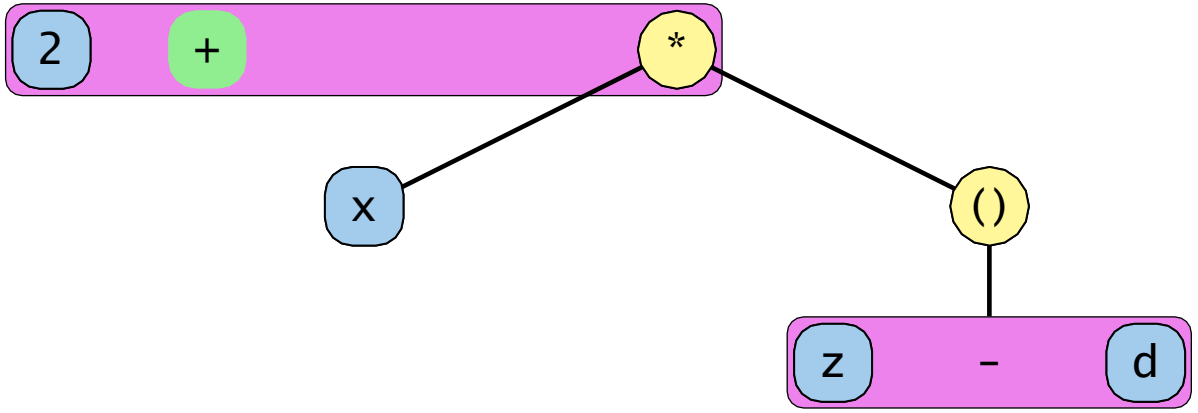
Beispiel: 2 + x * (z - d)



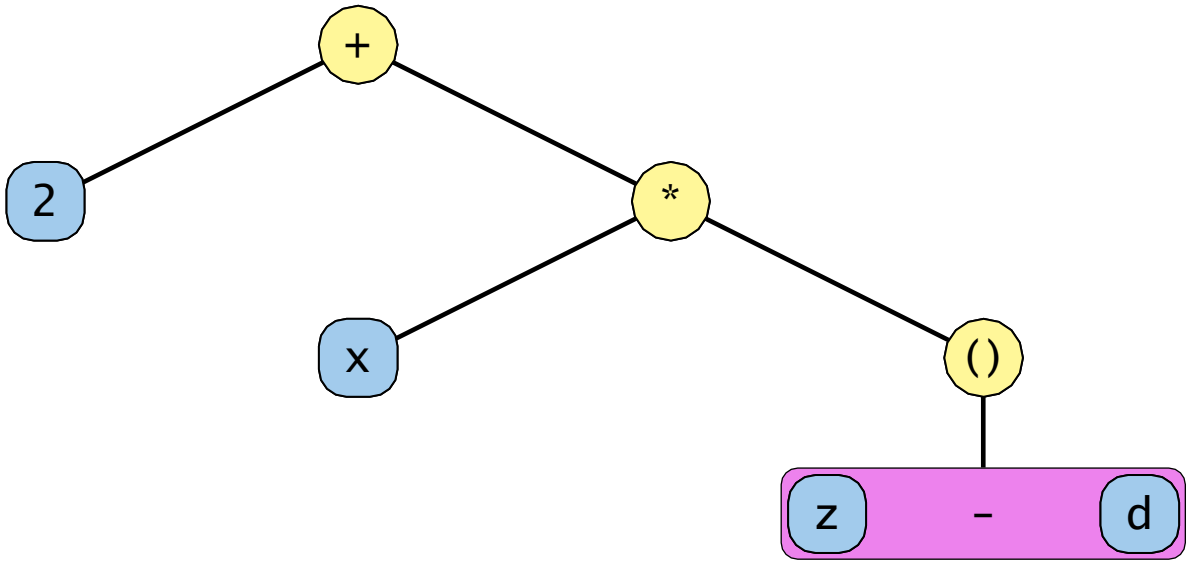
Beispiel: 2 + x * (z - d)



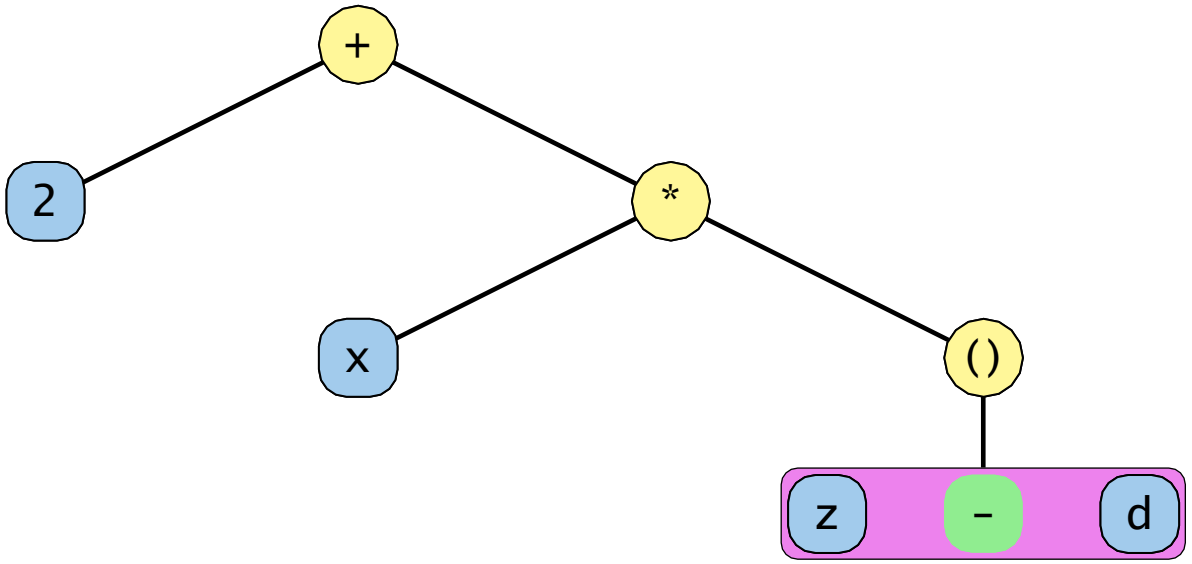
Beispiel: 2 + x * (z - d)



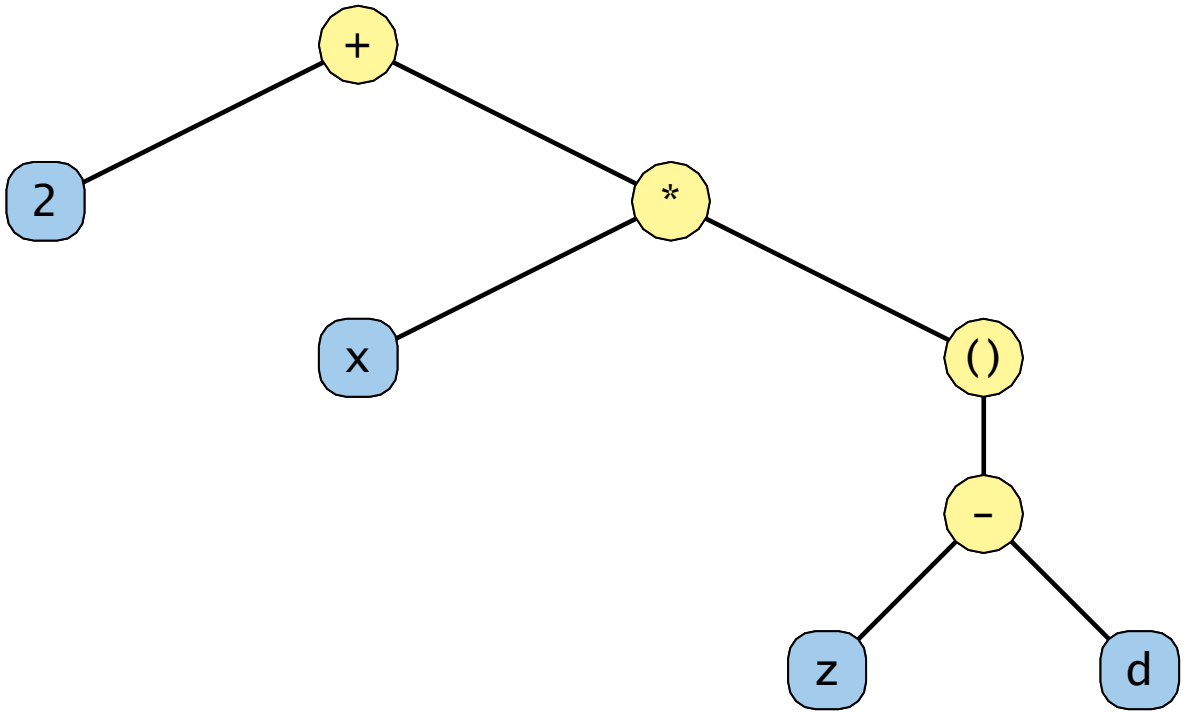
Beispiel: 2 + x * (z - d)



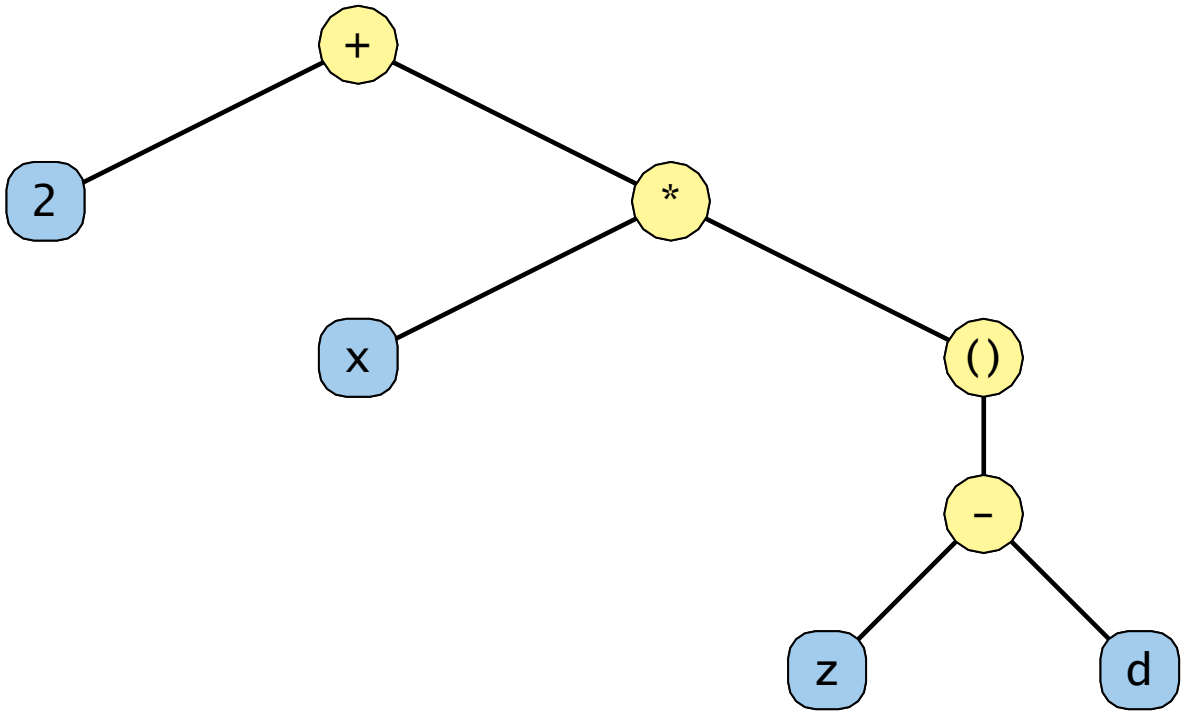
Beispiel: 2 + x * (z - d)



Beispiel: 2 + x * (z - d)

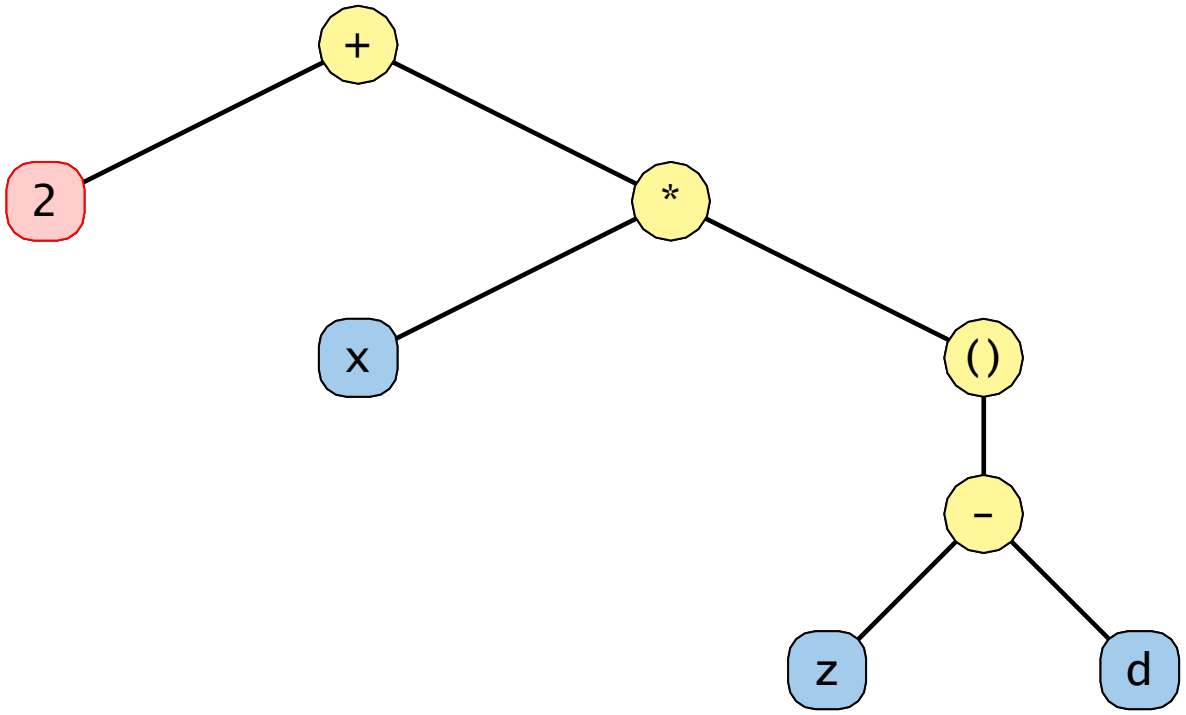


Beispiel: 2 + x * (z - d)



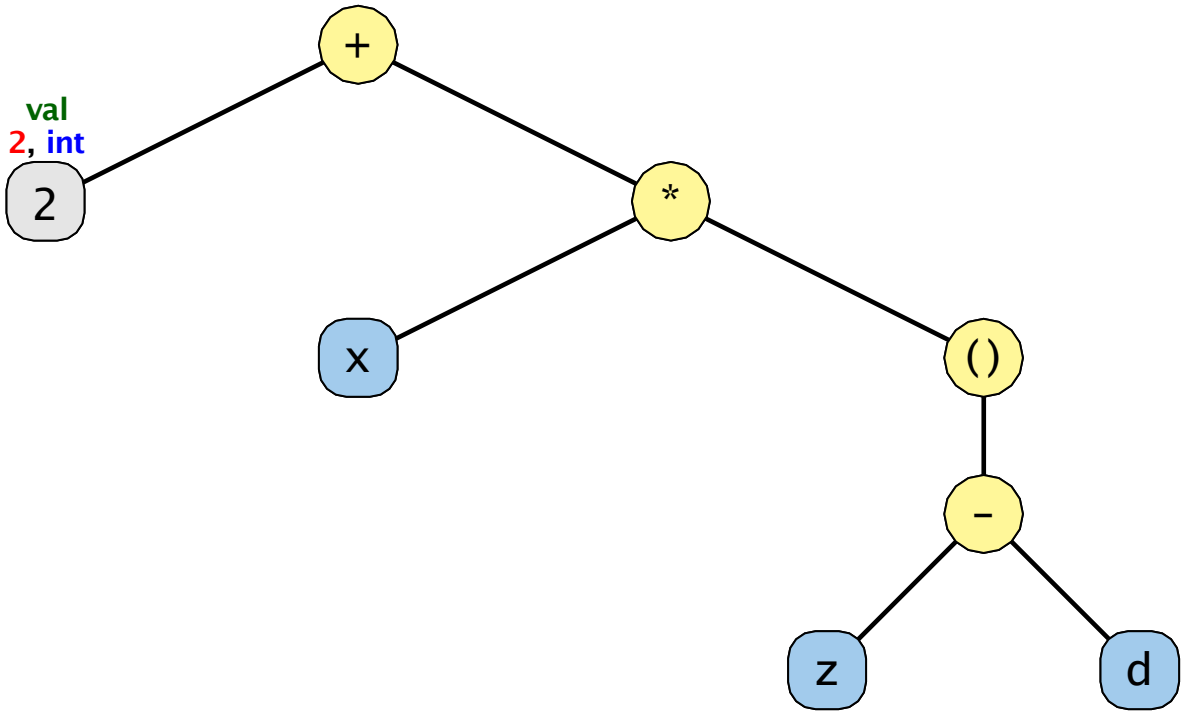
x -3 d 7 z 5

Beispiel: 2 + x * (z - d)



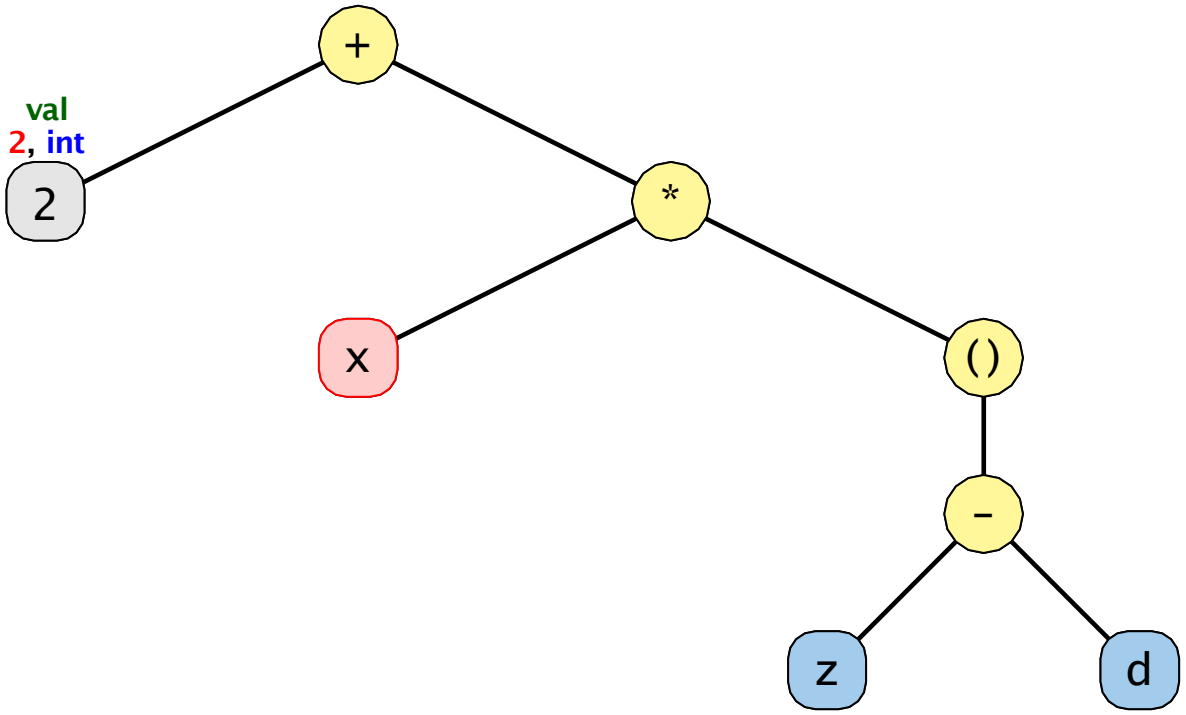
x -3 d 7 z 5

Beispiel: 2 + x * (z - d)



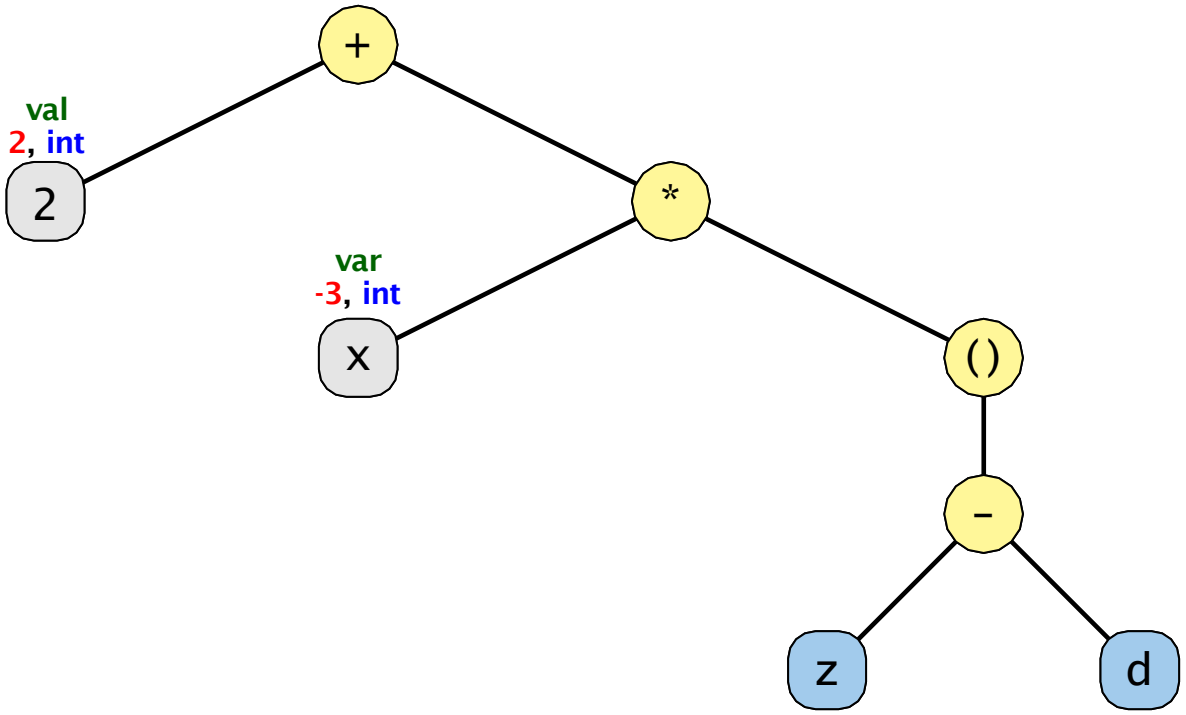
x -3 d 7 z 5

Beispiel: 2 + x * (z - d)



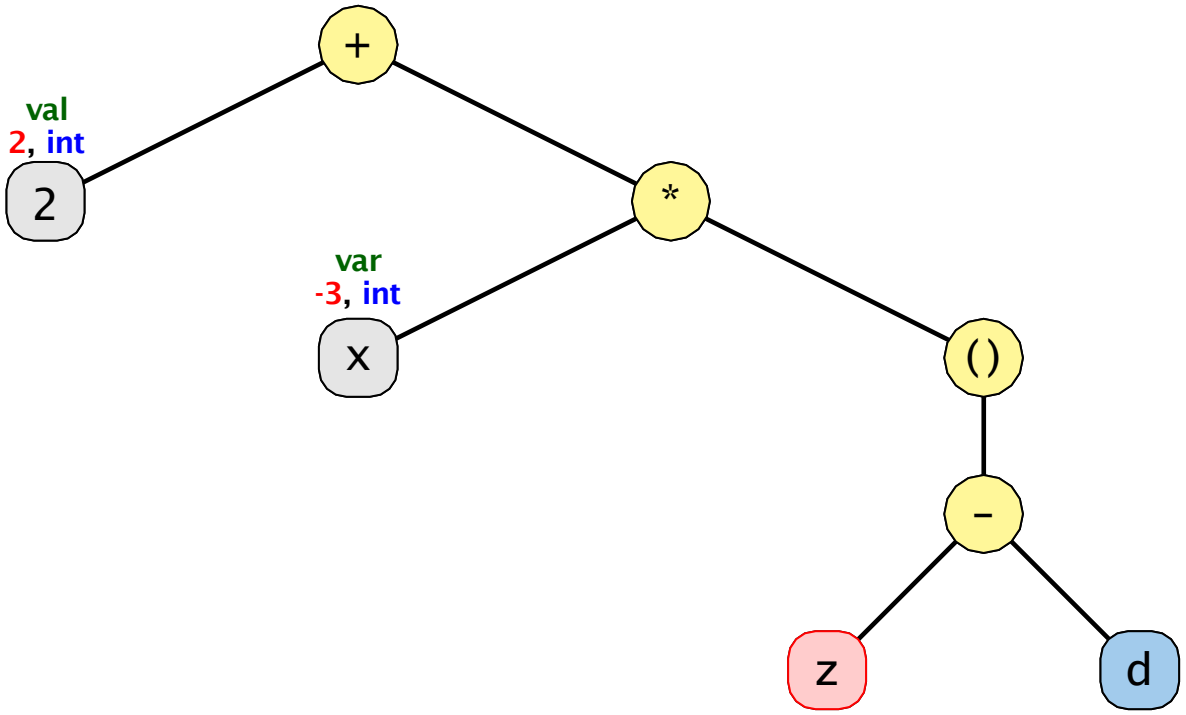
x -3 d 7 z 5

Beispiel: 2 + x * (z - d)



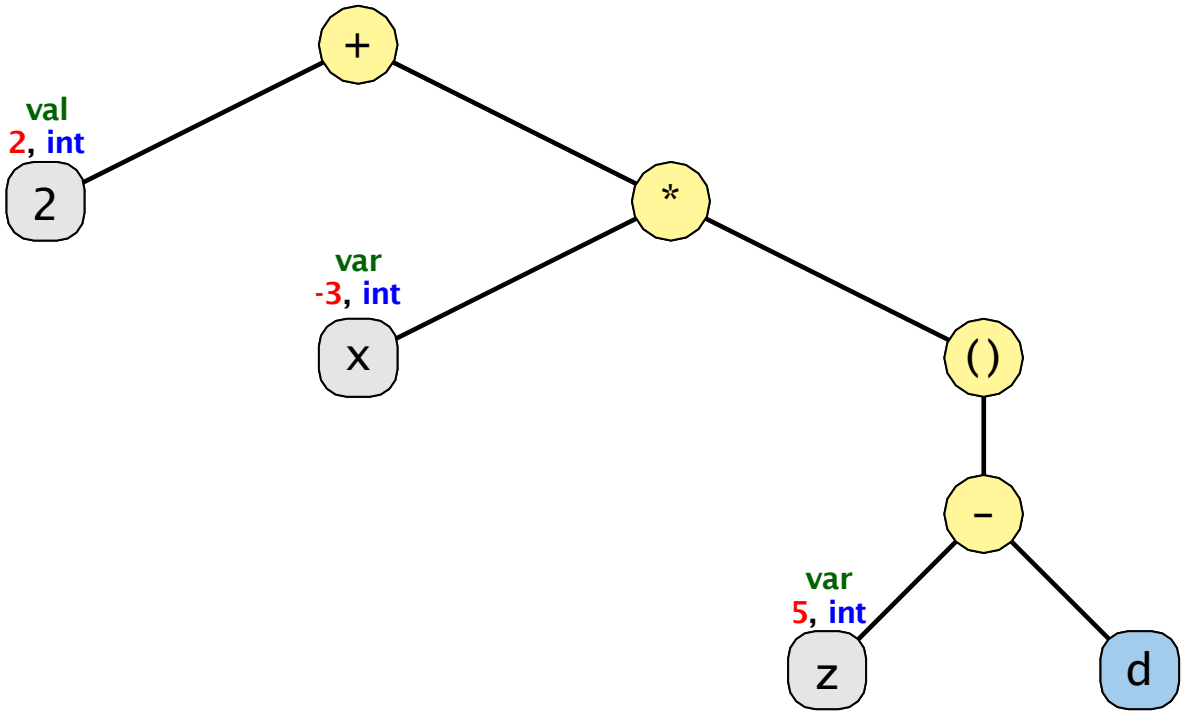
x -3 d 7 z 5

Beispiel: 2 + x * (z - d)



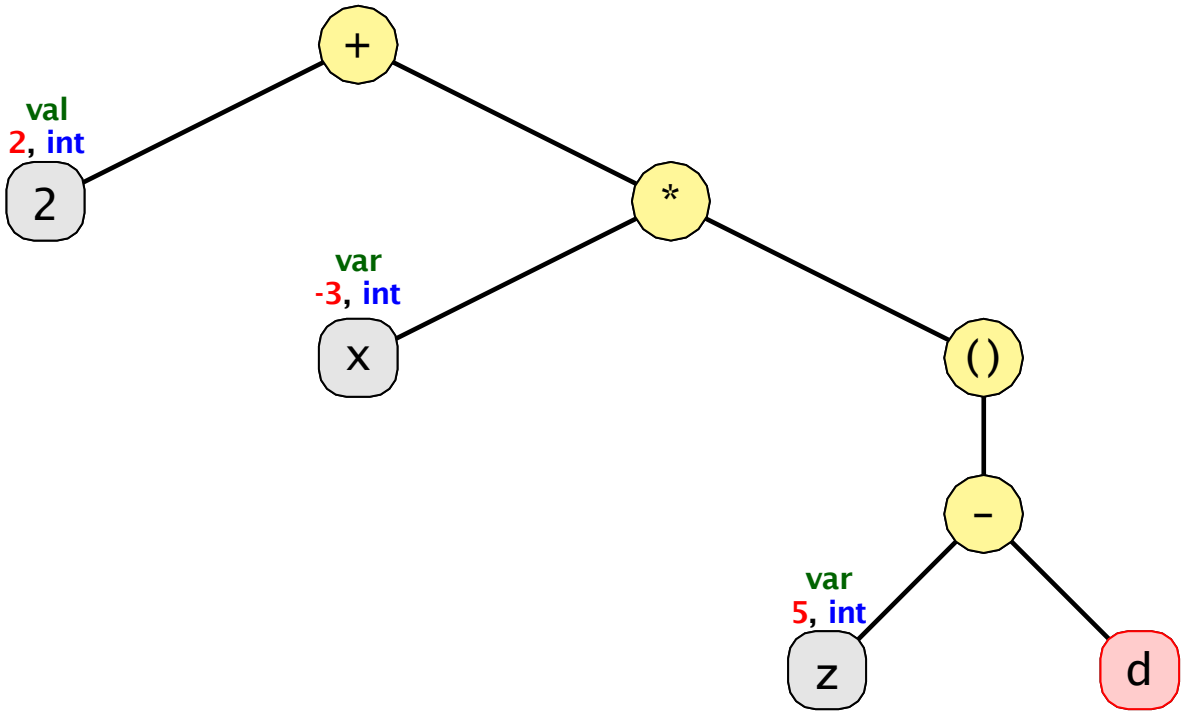
x -3 d 7 z 5

Beispiel: 2 + x * (z - d)



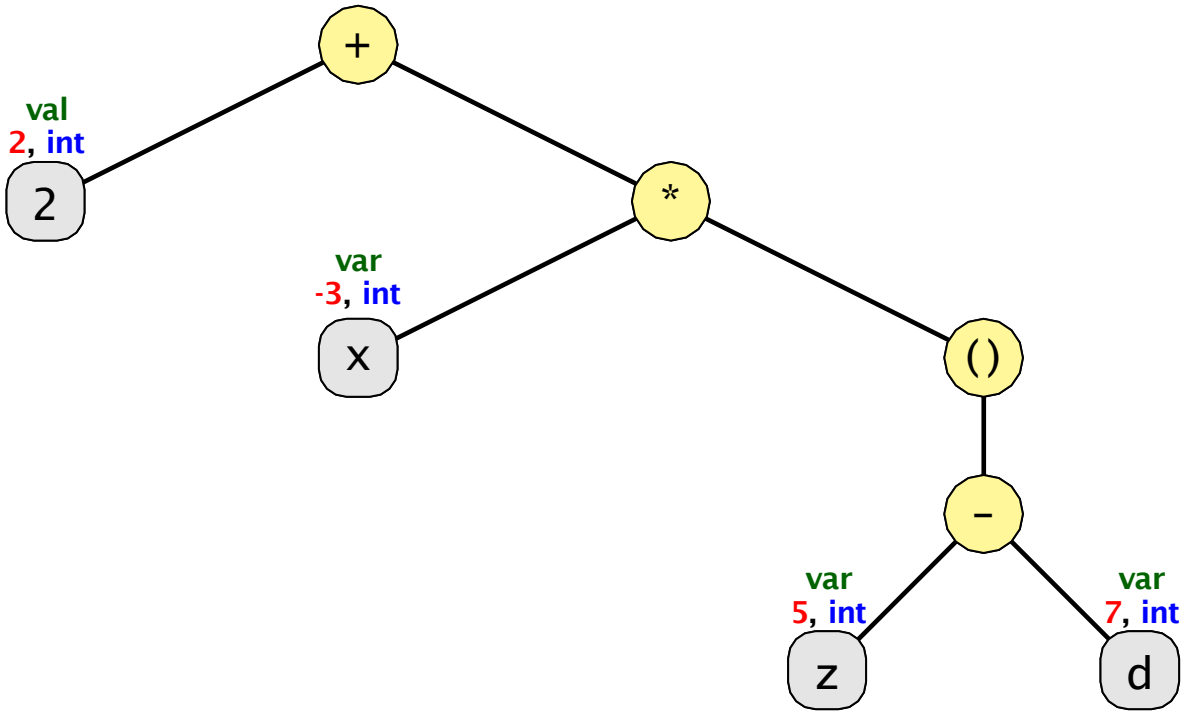
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



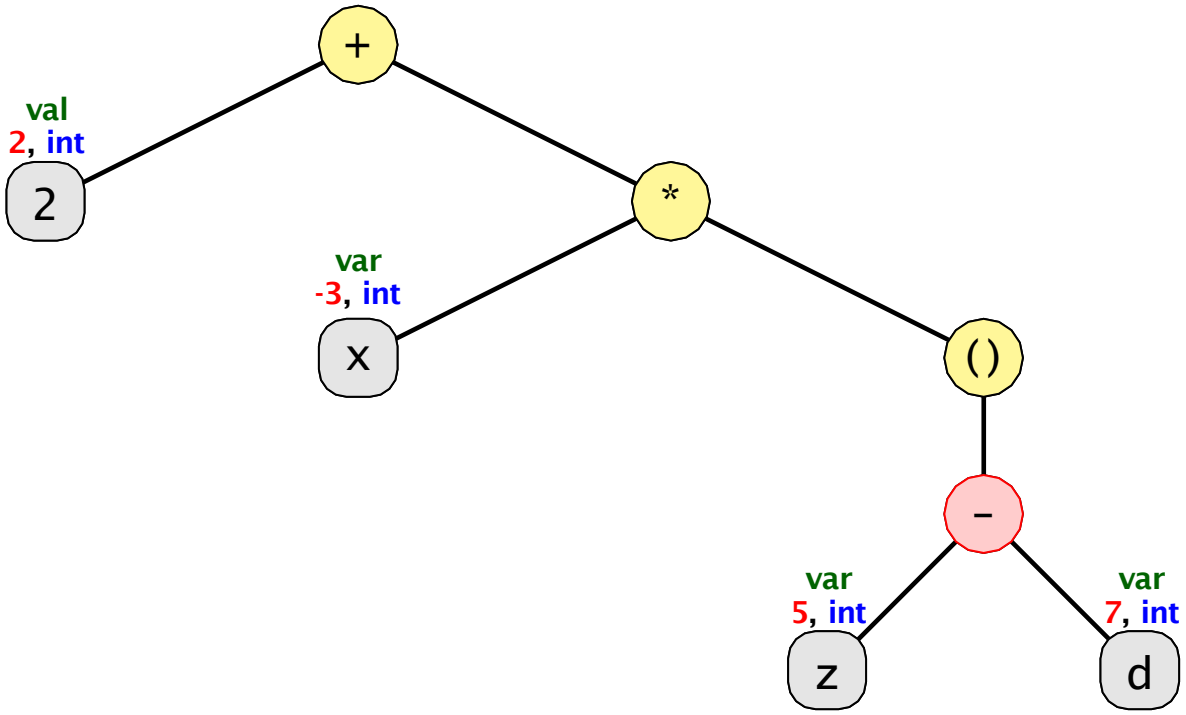
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



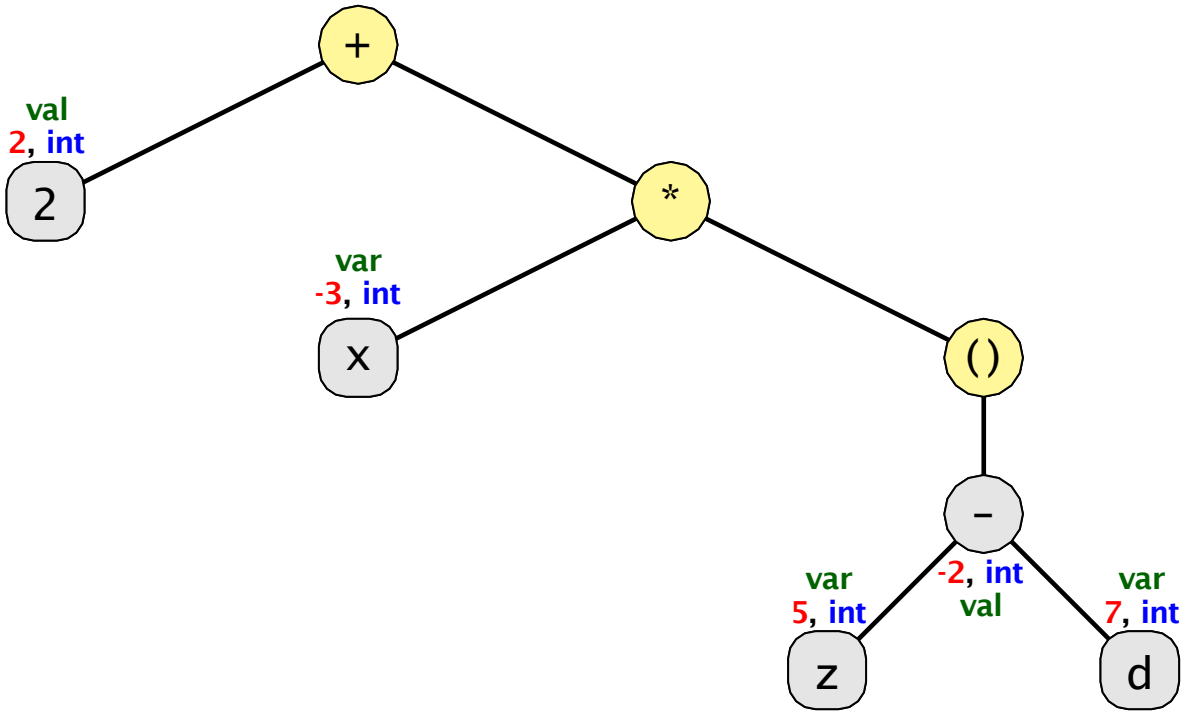
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



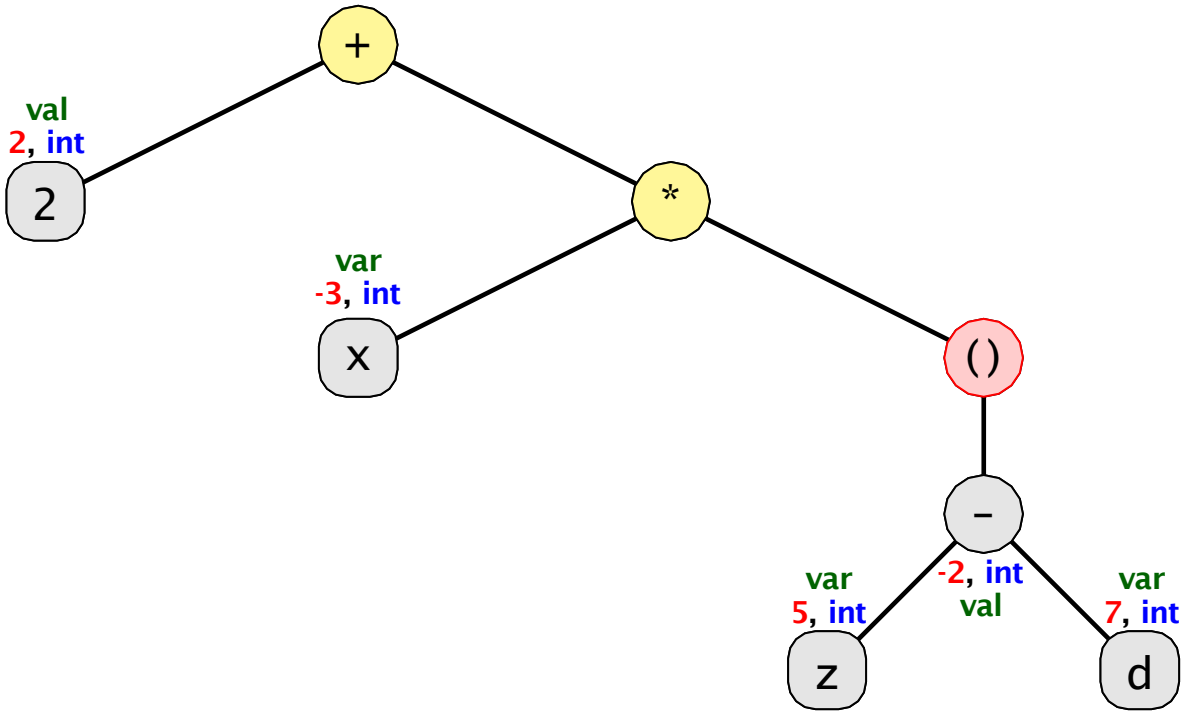
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



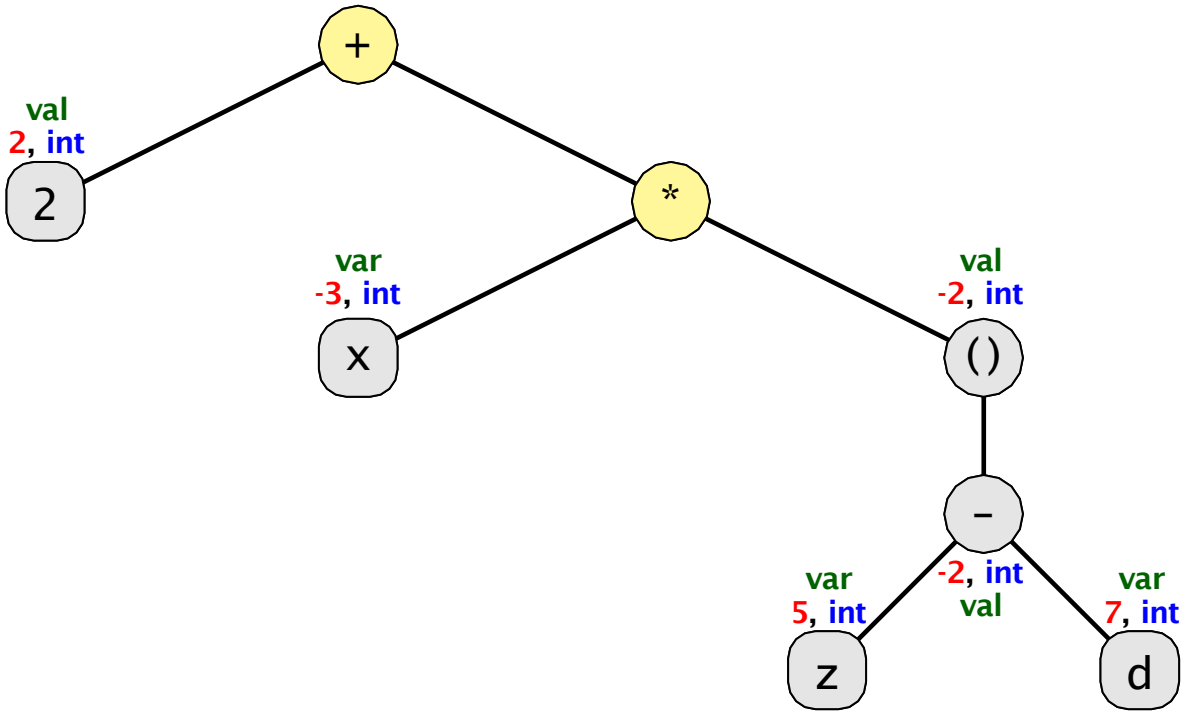
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



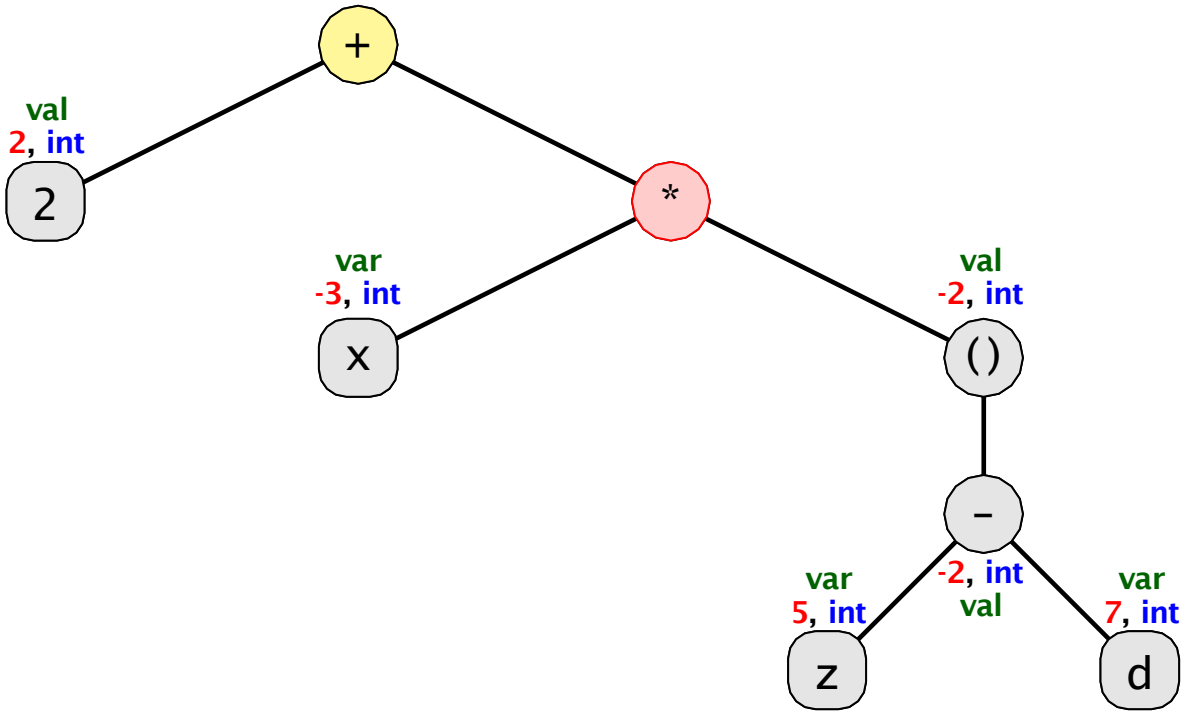
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



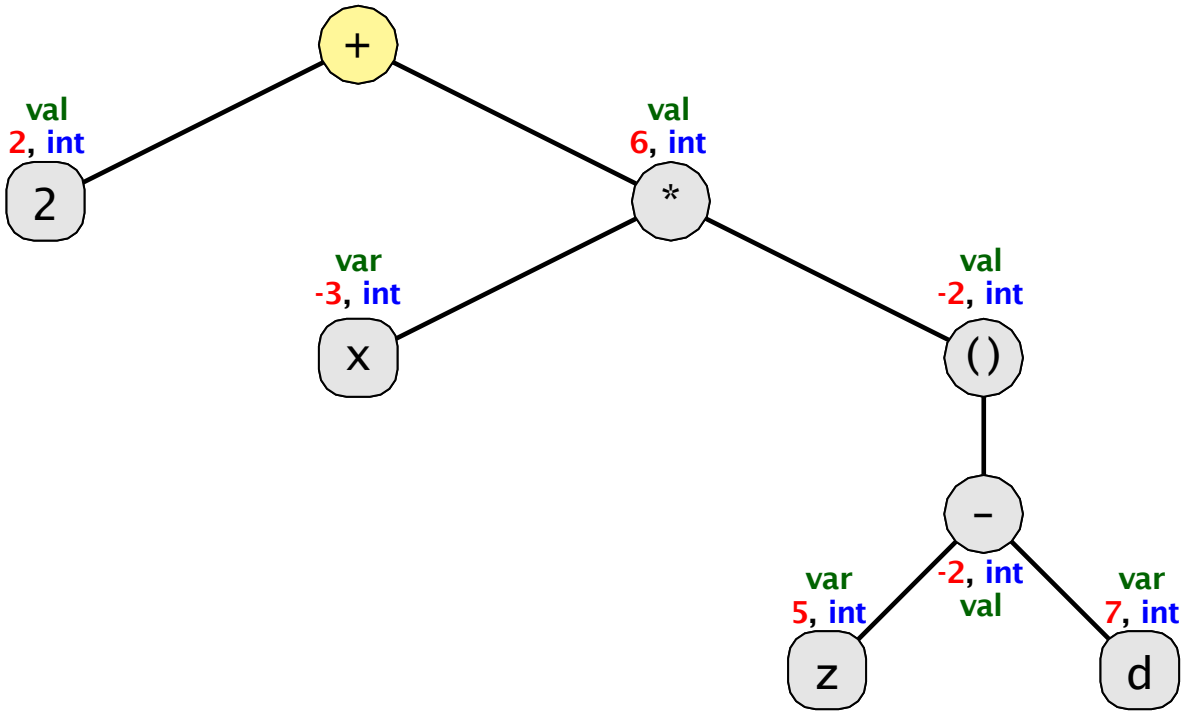
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



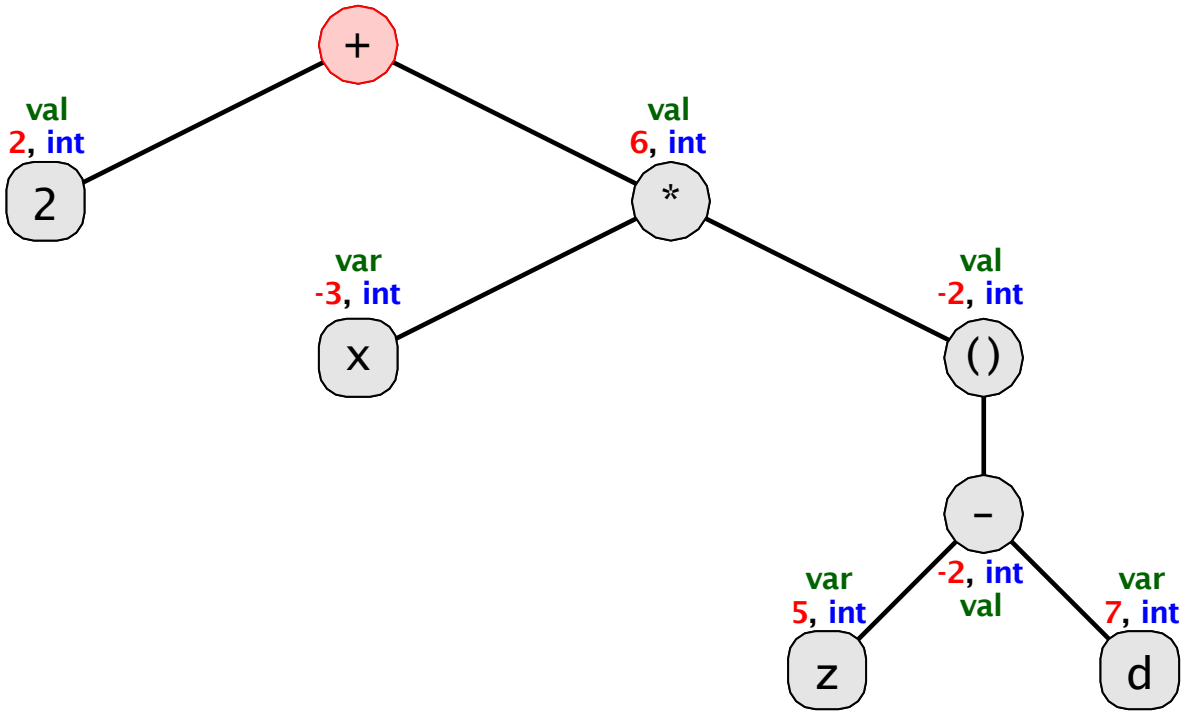
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



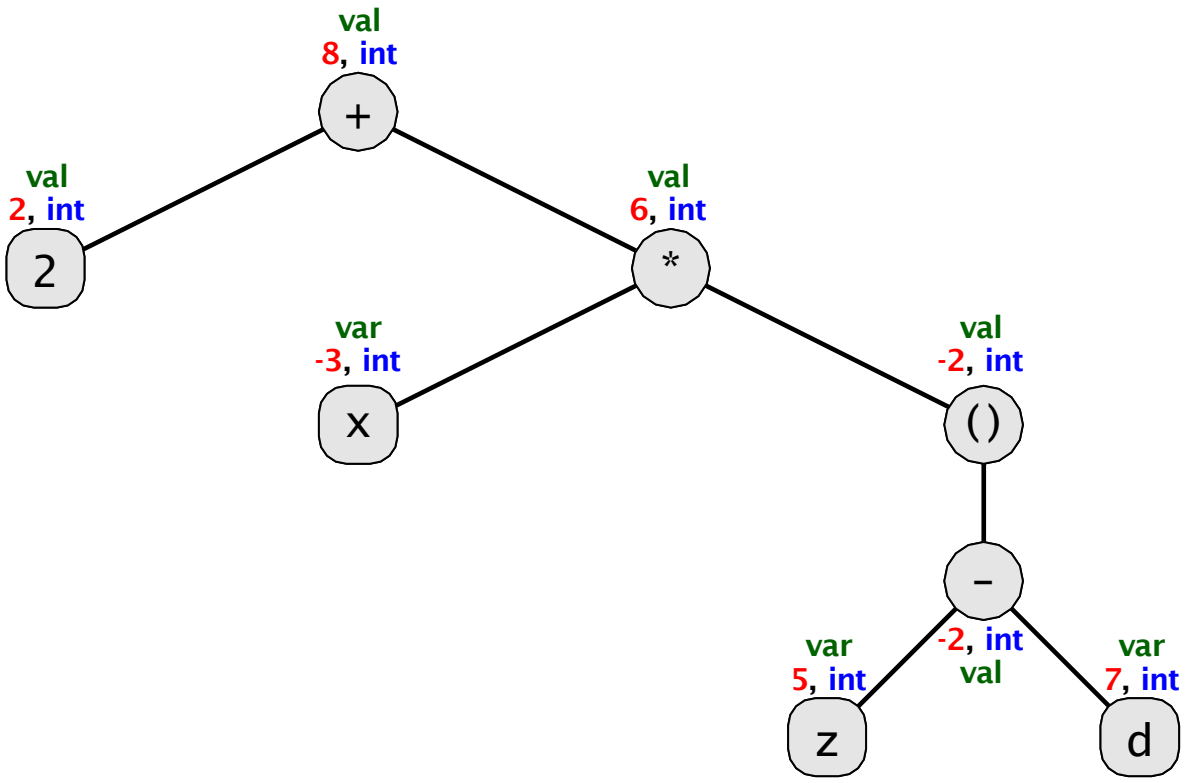
x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)



x | -3 | d | 7 | z | 5 |

Beispiel: 2 + x * (z - d)

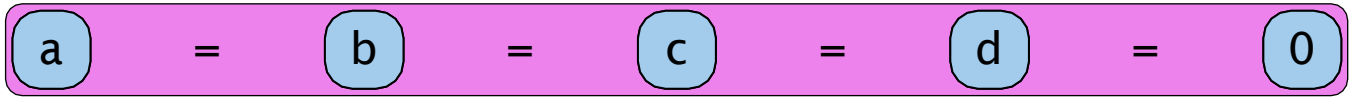


x | -3 | d | 7 | z | 5 |

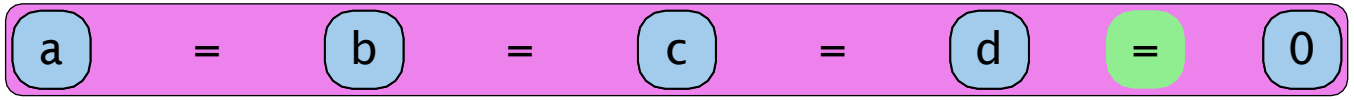
Beispiel: a = b = c = d = 0

a = b = c = d = 0

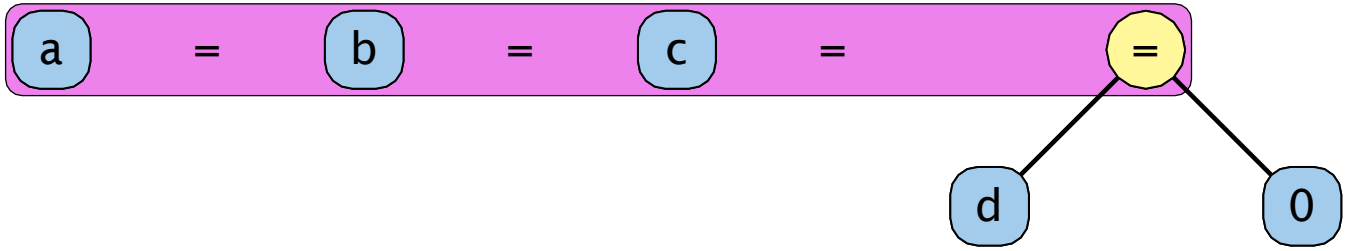
Beispiel: a = b = c = d = 0



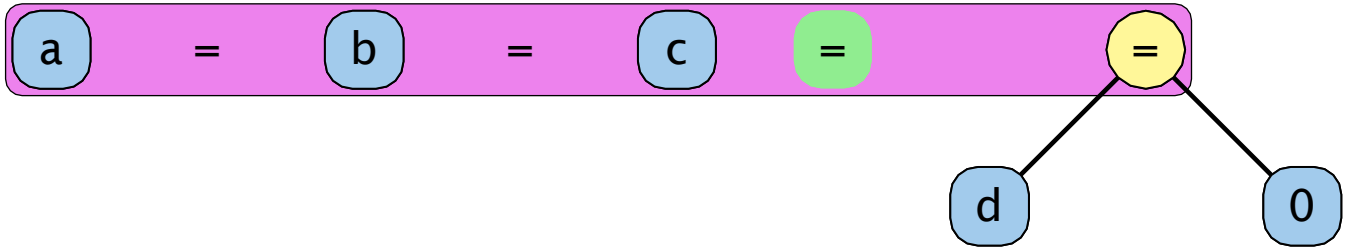
Beispiel: a = b = c = d = 0



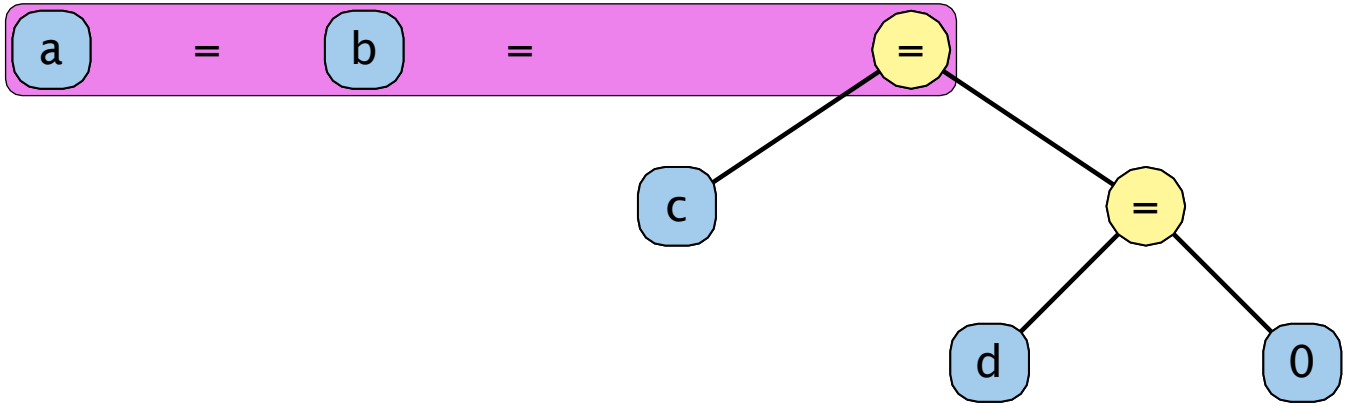
Beispiel: a = b = c = d = 0



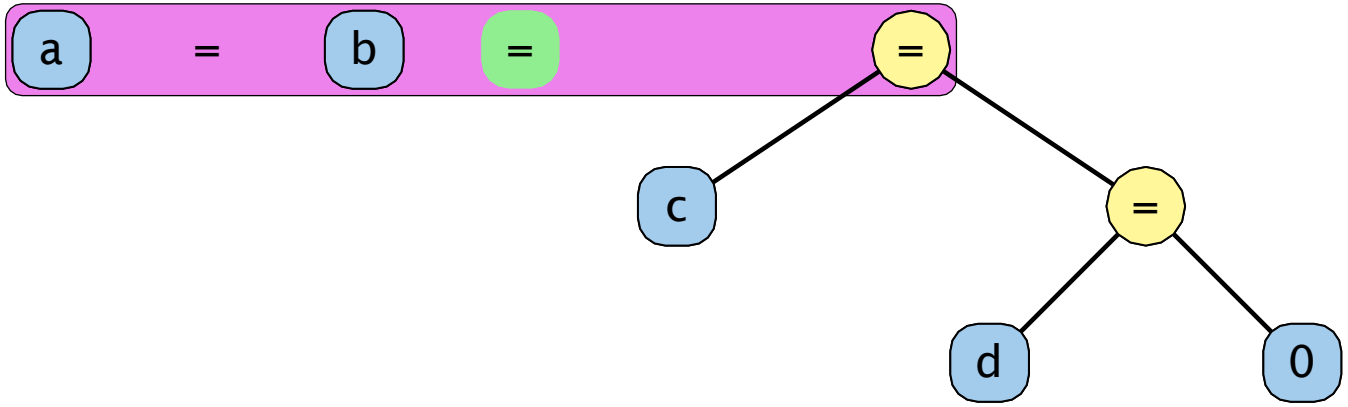
Beispiel: a = b = c = d = 0



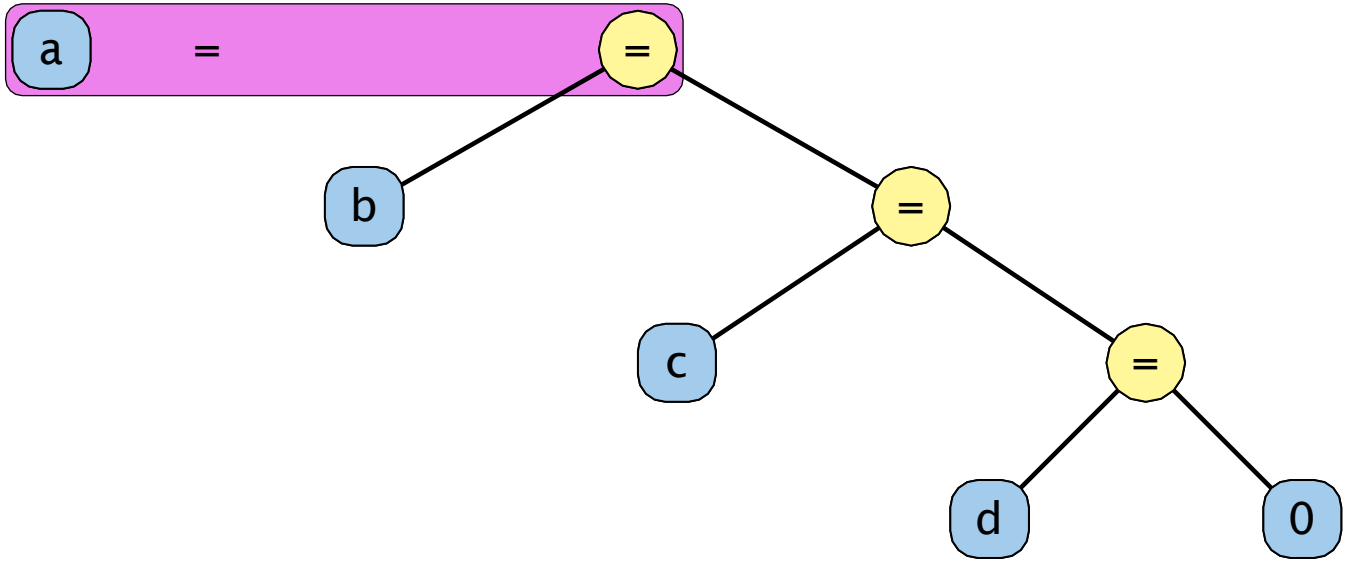
Beispiel: a = b = c = d = 0



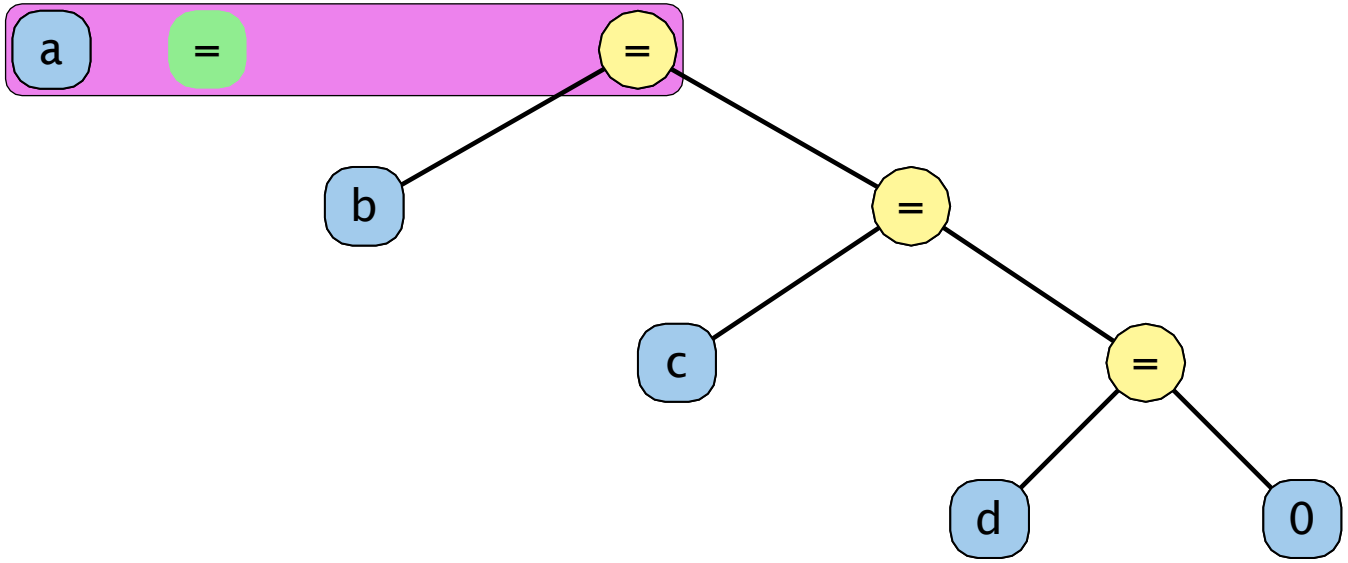
Beispiel: a = b = c = d = 0



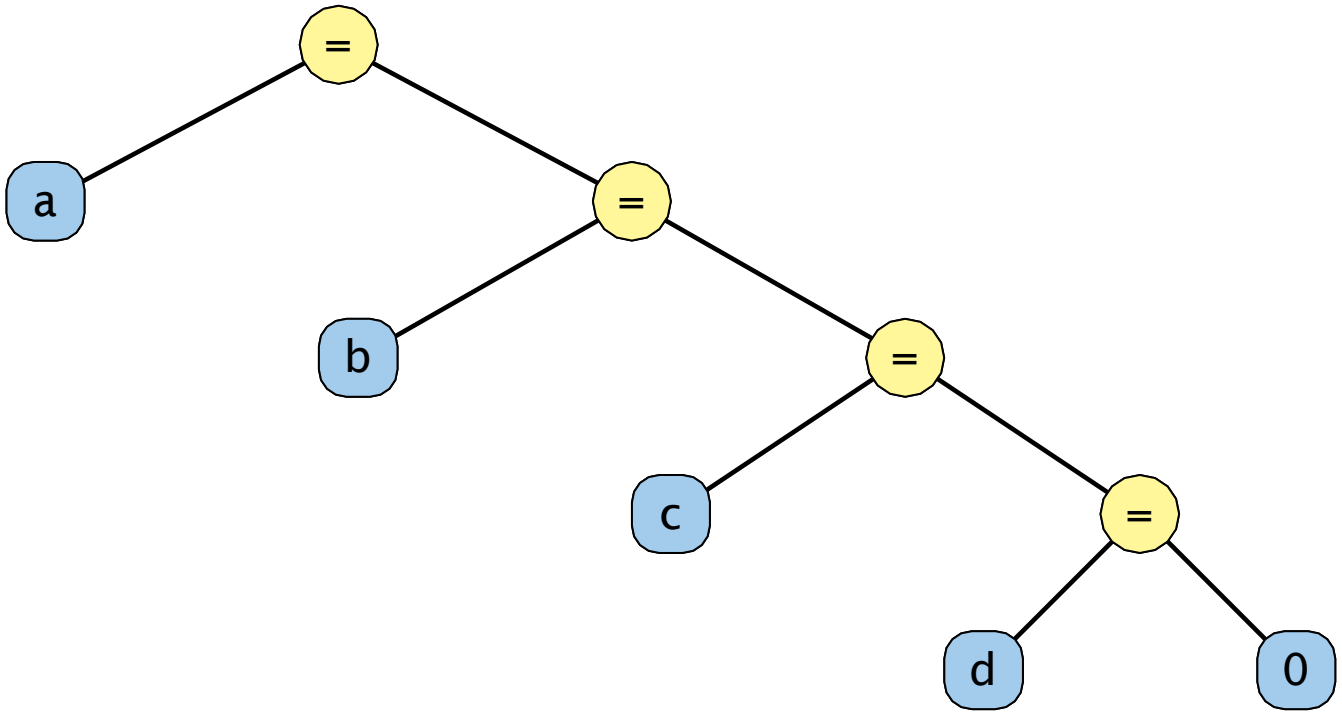
Beispiel: a = b = c = d = 0



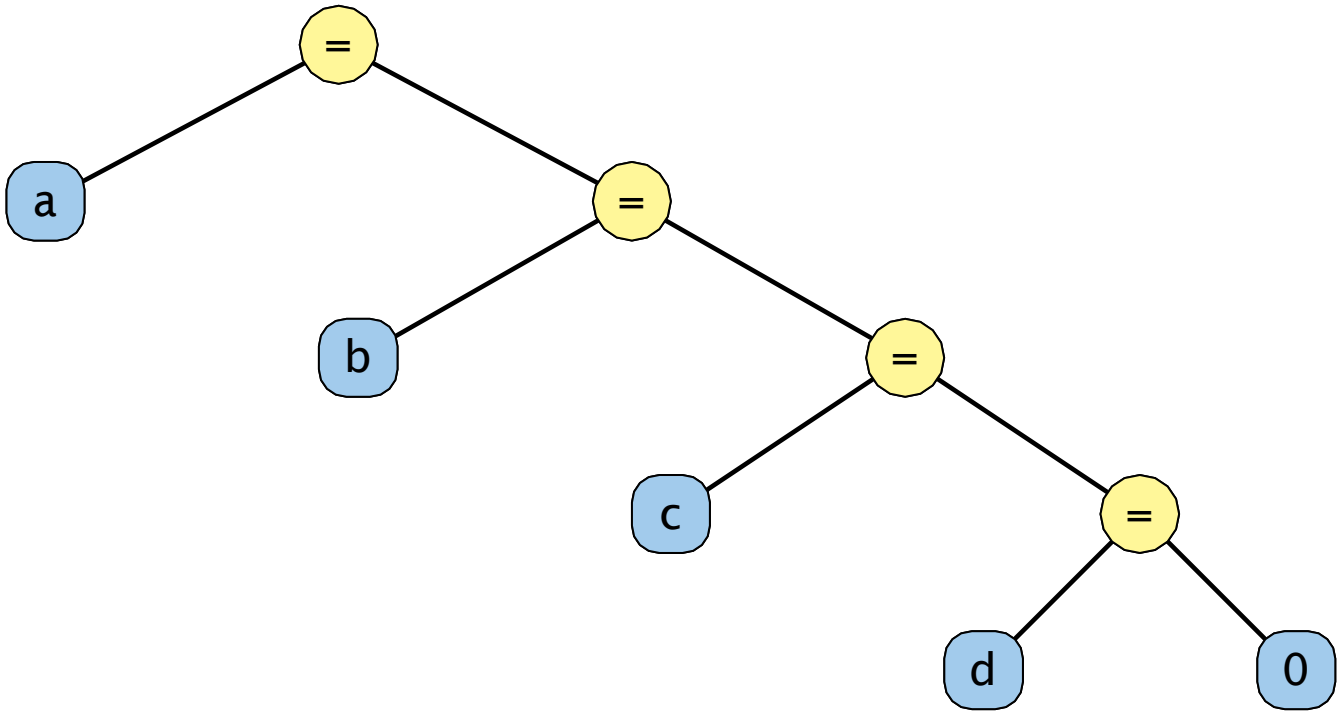
Beispiel: a = b = c = d = 0



Beispiel: a = b = c = d = 0

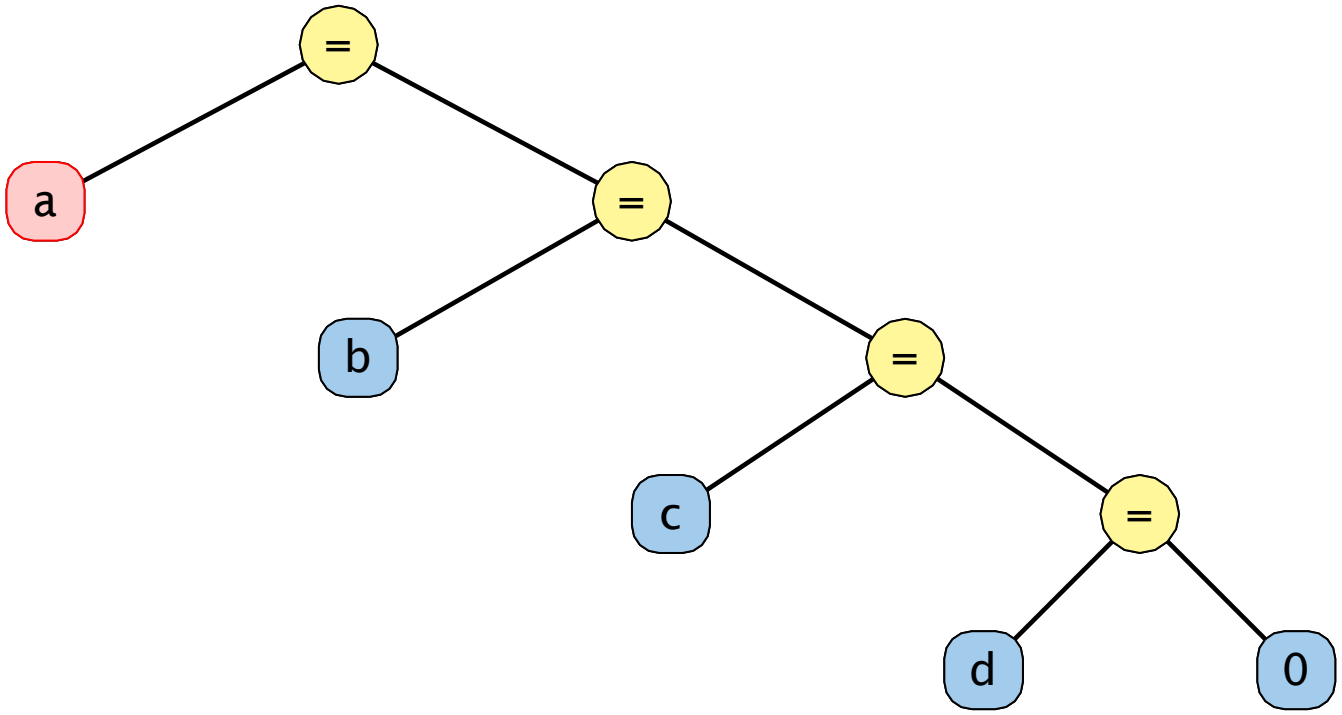


Beispiel: a = b = c = d = 0



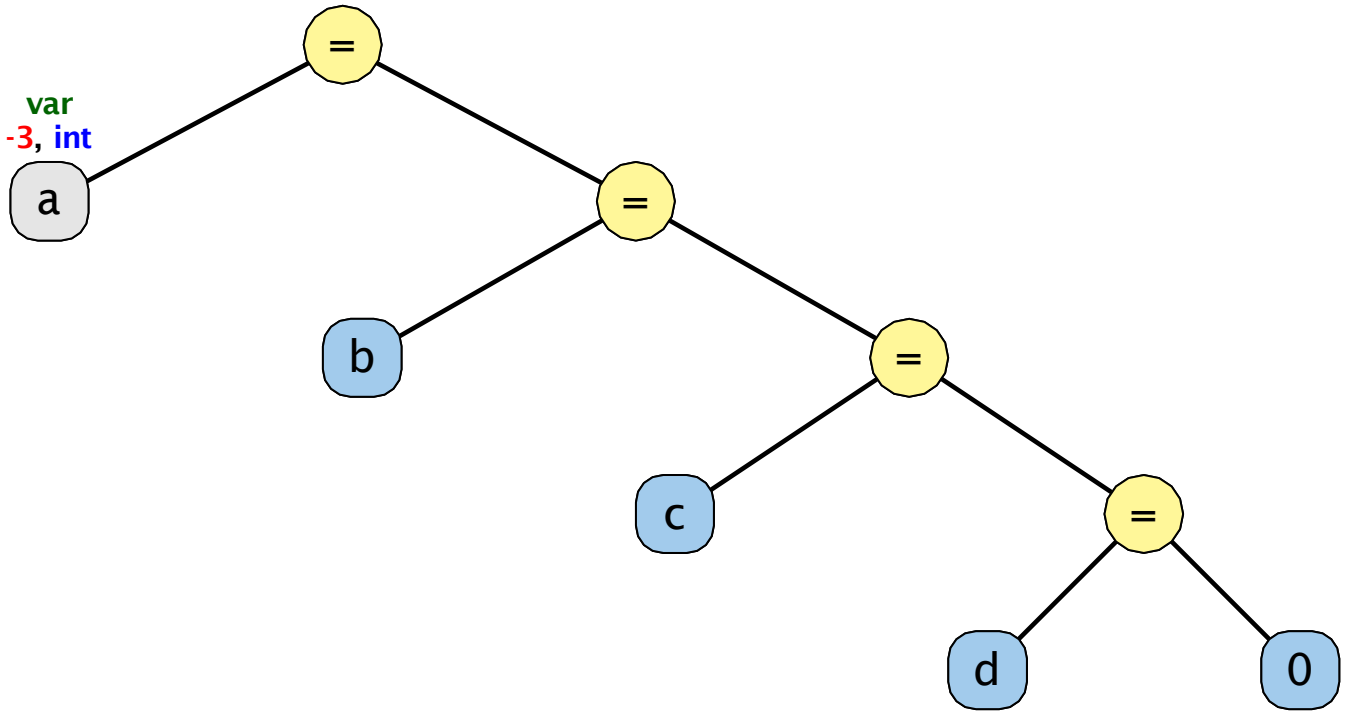
a -3 b 7 c 5 d 2

Beispiel: a = b = c = d = 0



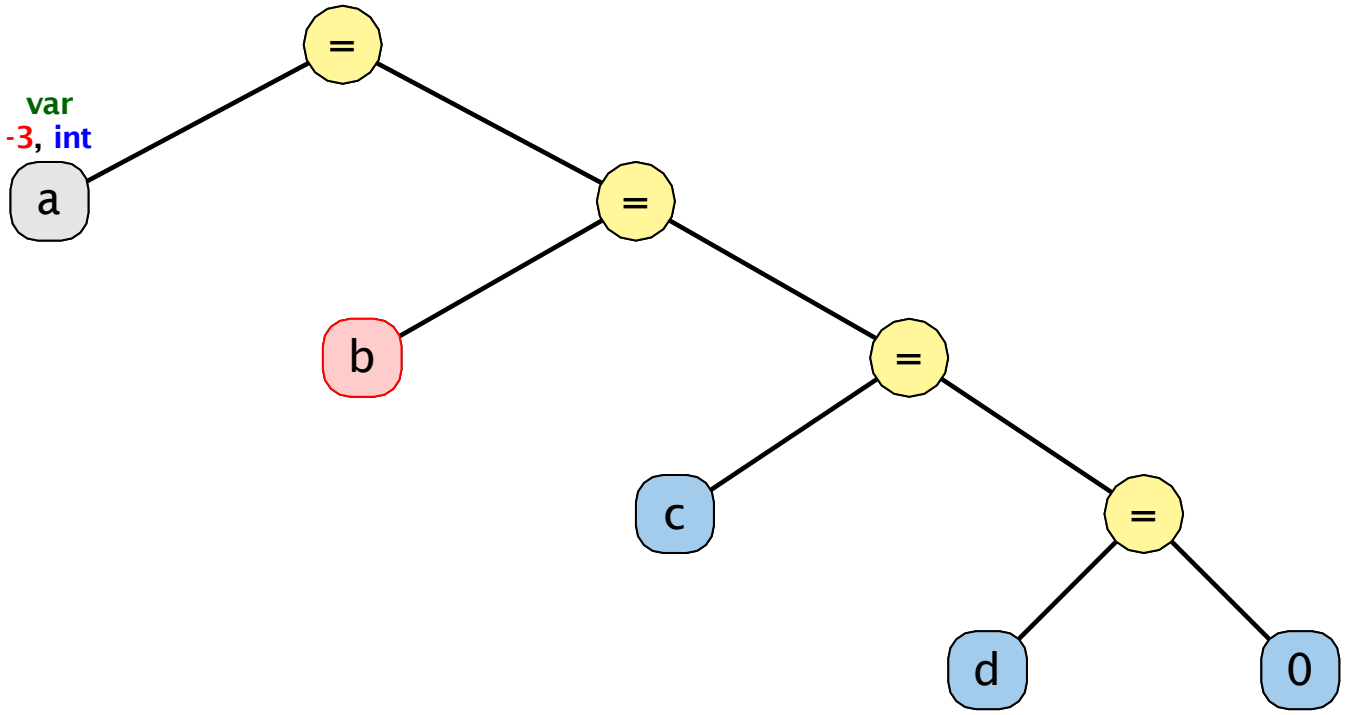
a -3 b 7 c 5 d 2

Beispiel: a = b = c = d = 0



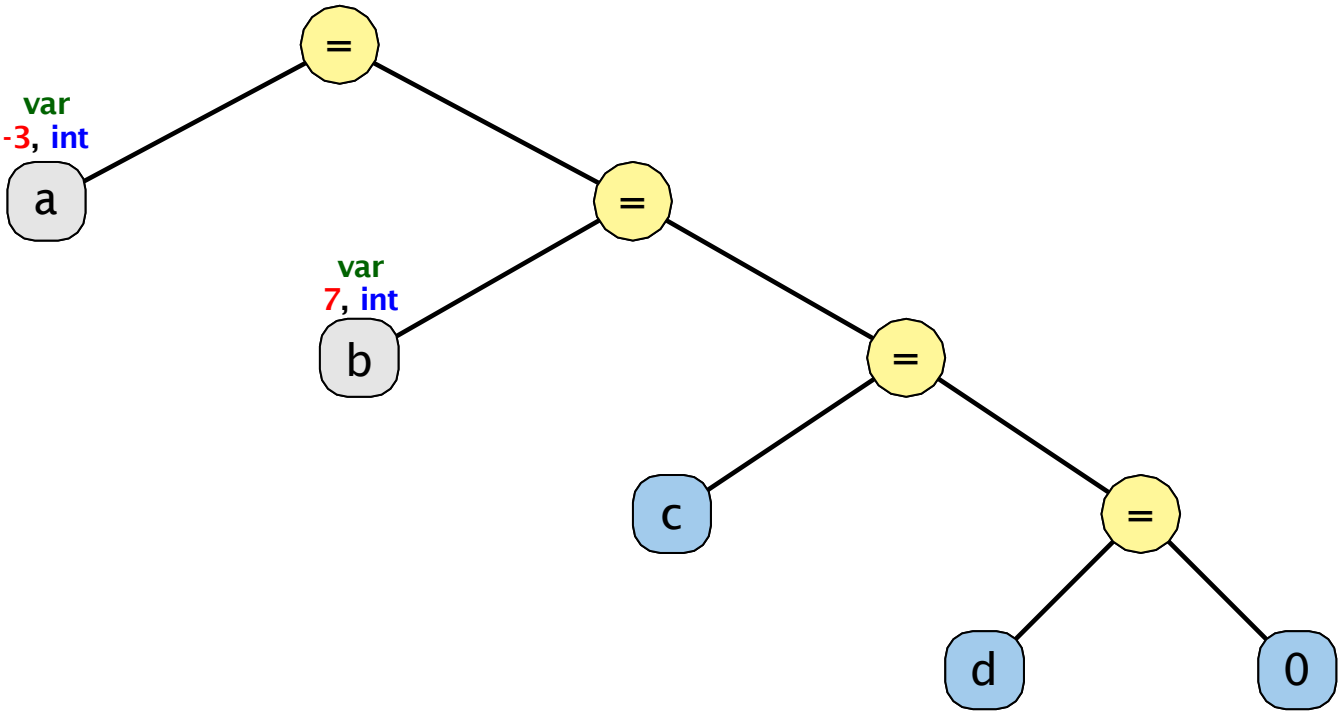
a -3 b 7 c 5 d 2

Beispiel: a = b = c = d = 0



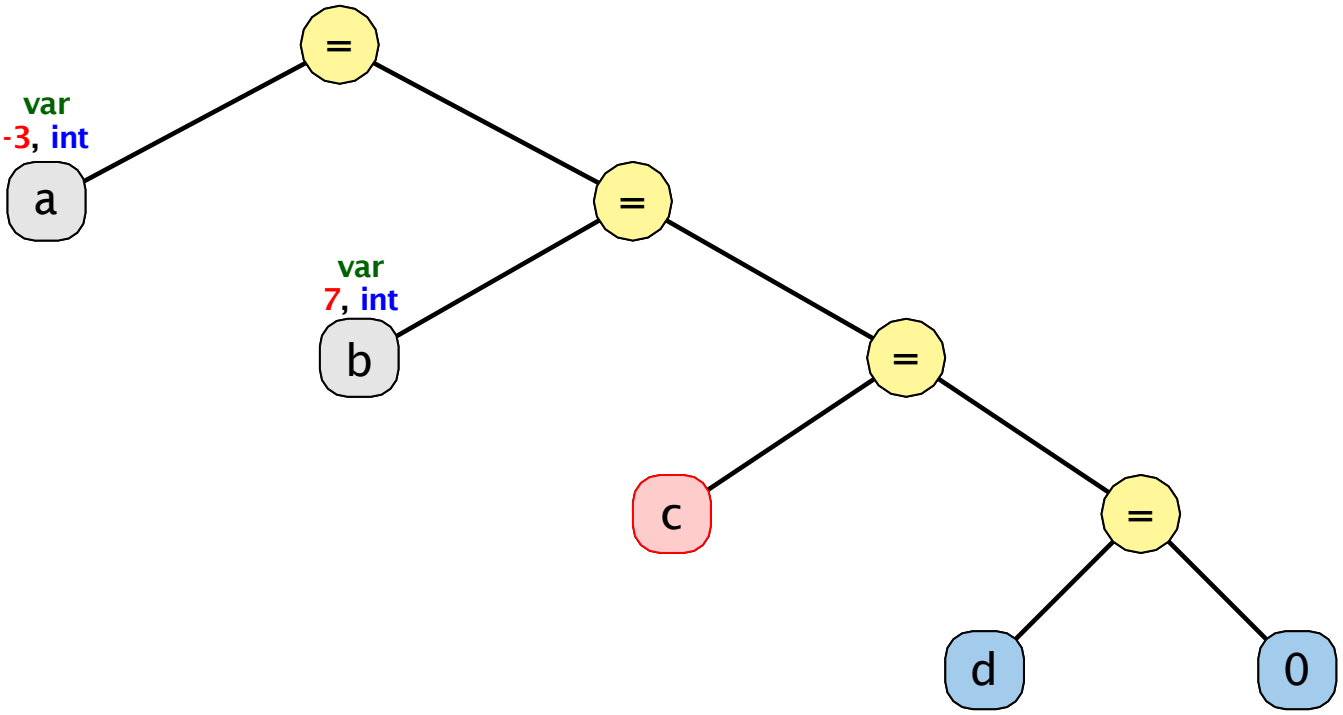
a -3 b 7 c 5 d 2

Beispiel: a = b = c = d = 0



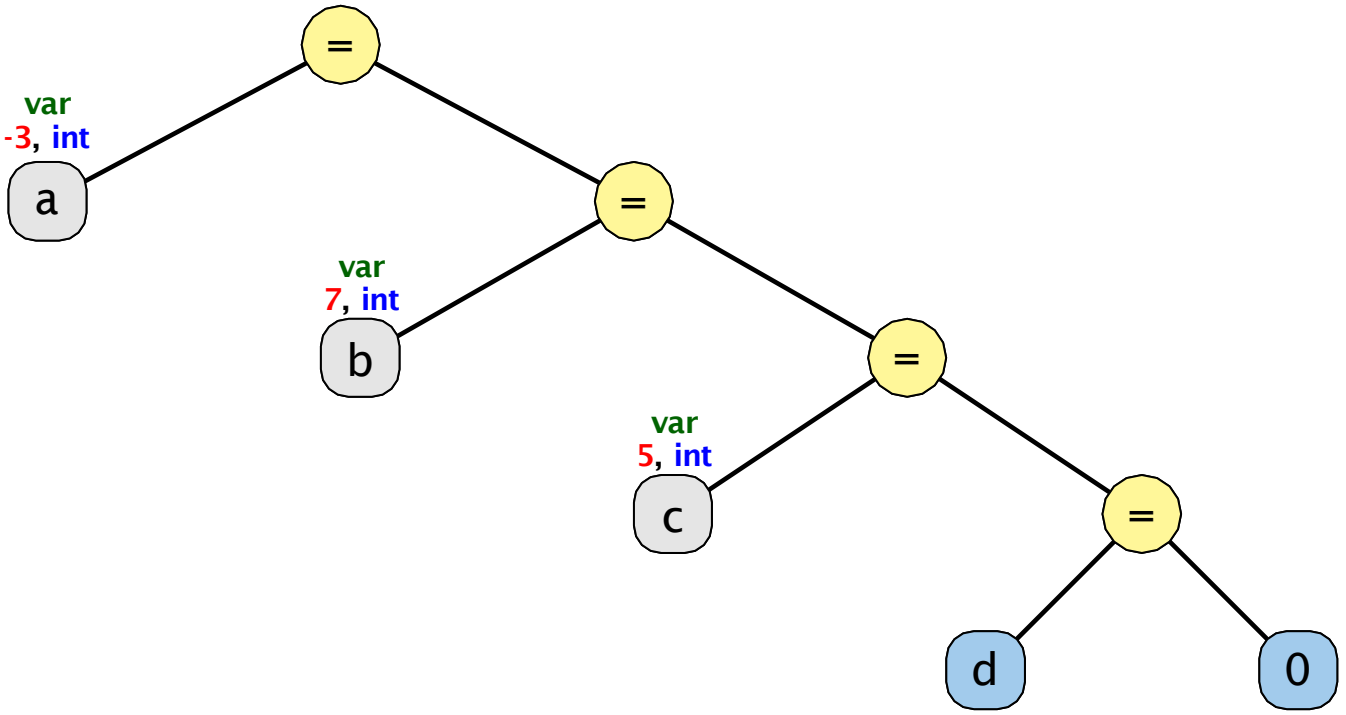
a -3 b 7 c 5 d 2

Beispiel: a = b = c = d = 0



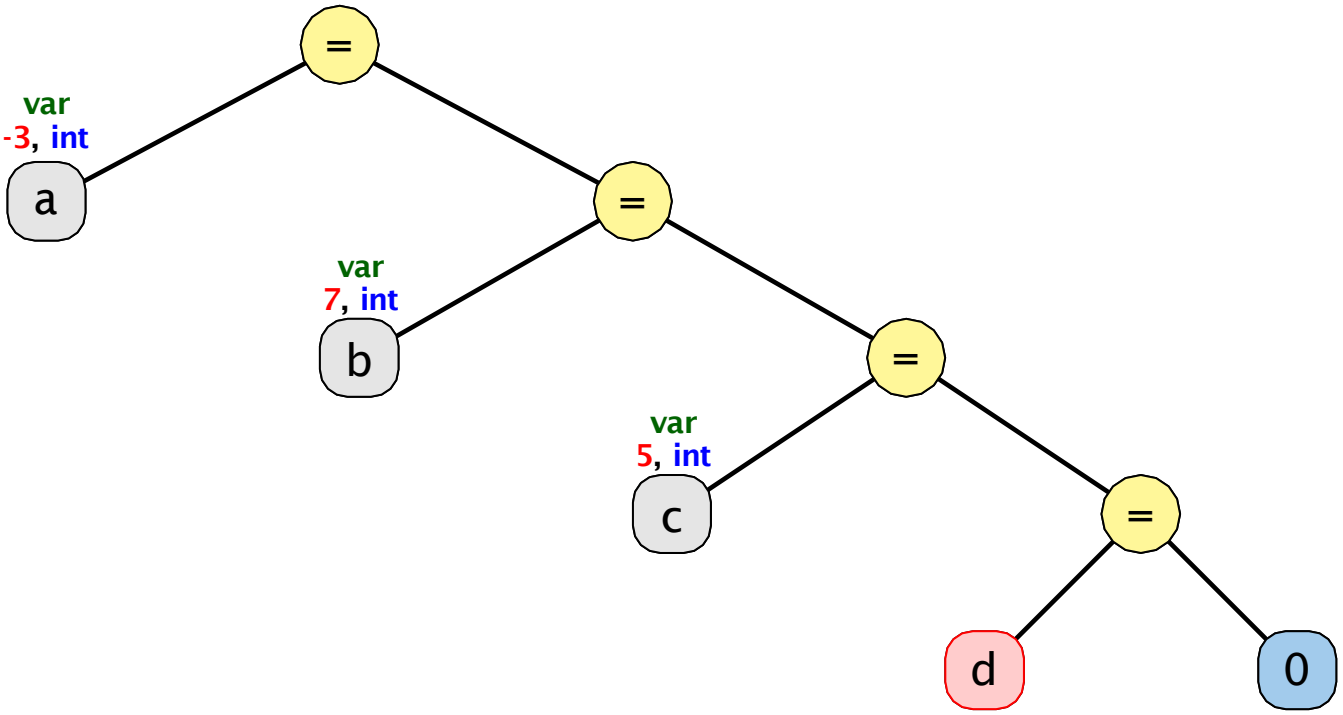
a -3 b 7 c 5 d 2

Beispiel: a = b = c = d = 0



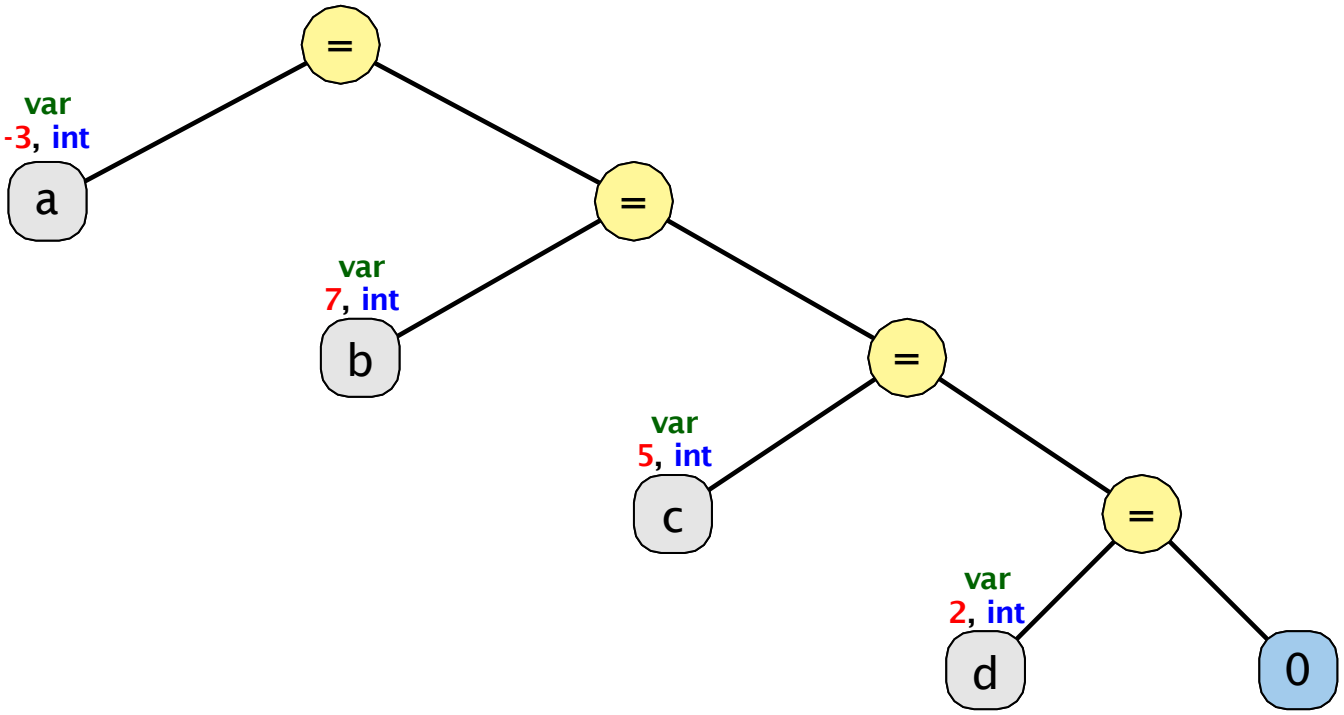
a -3 b 7 c 5 d 2

Beispiel: a = b = c = d = 0



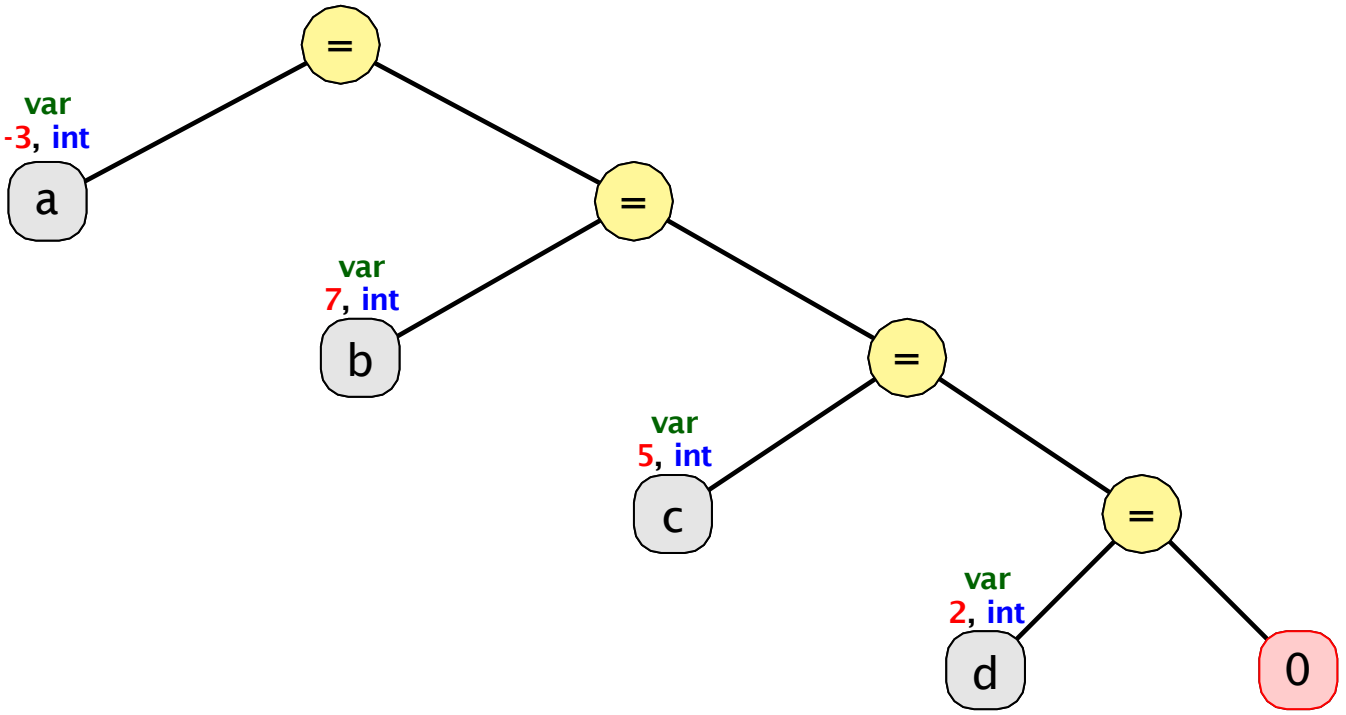
a	b	c	d
-3	7	5	2

Beispiel: a = b = c = d = 0



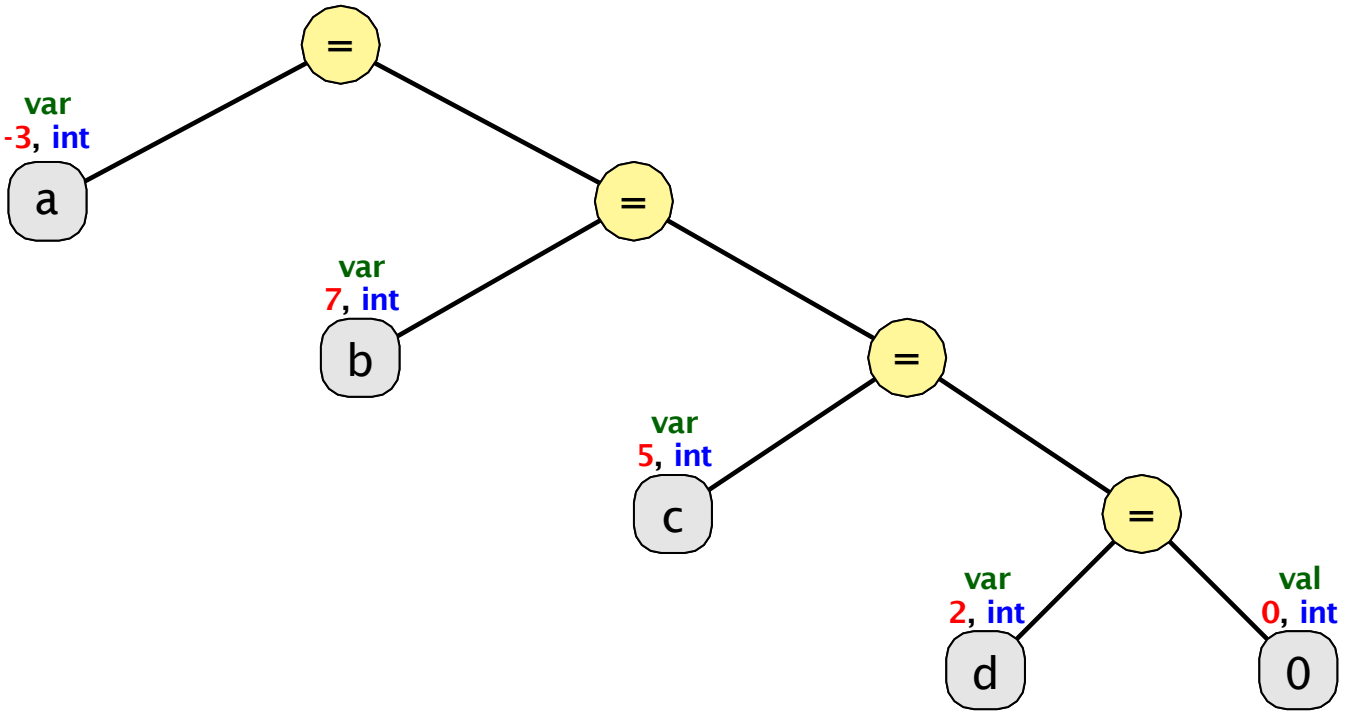
a	b	c	d
-3	7	5	2

Beispiel: a = b = c = d = 0



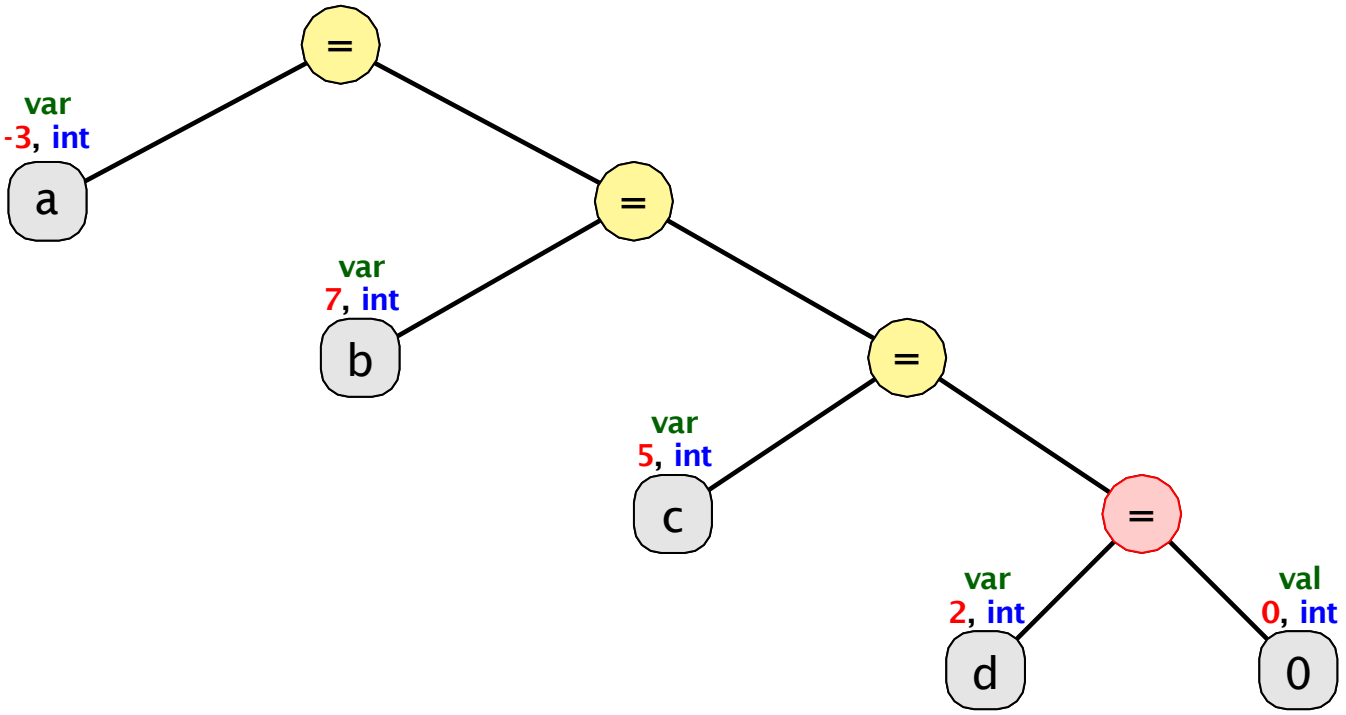
a	b	c	d
-3	7	5	2

Beispiel: a = b = c = d = 0



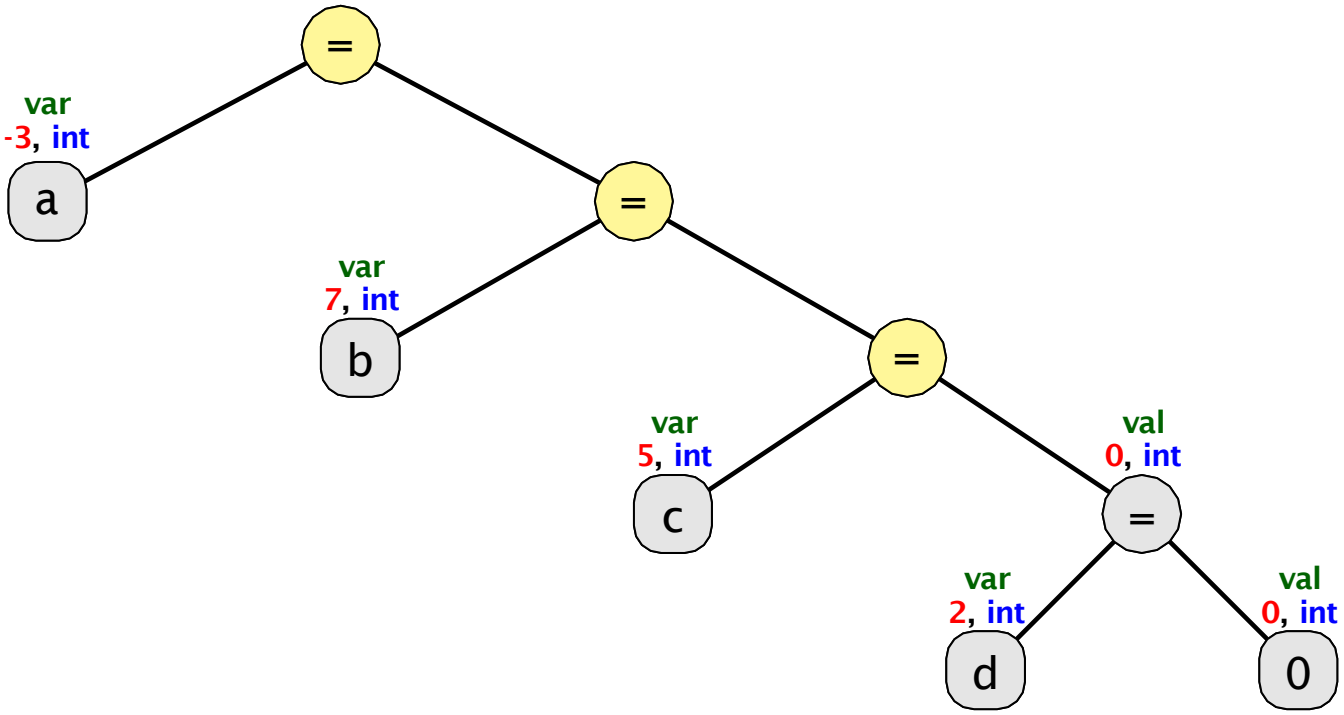
a	b	c	d
-3	7	5	2

Beispiel: a = b = c = d = 0



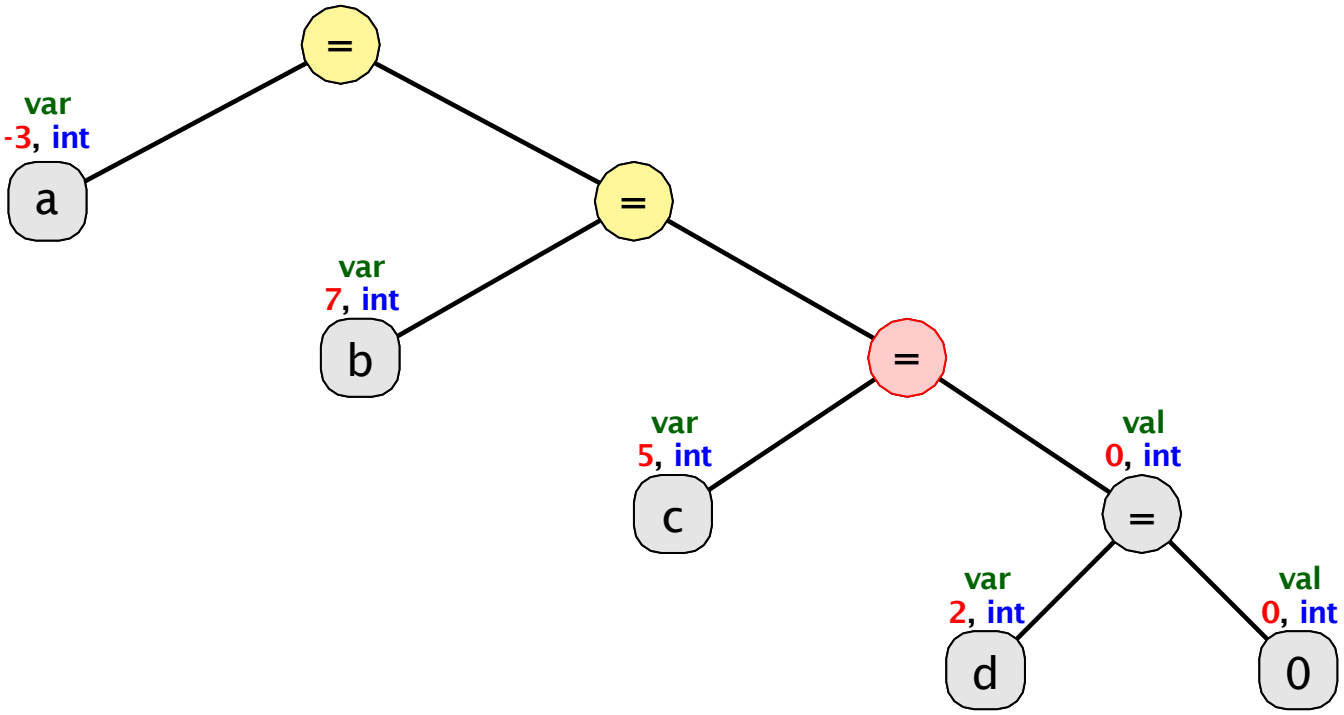
a	b	c	d
-3	7	5	2

Beispiel: a = b = c = d = 0



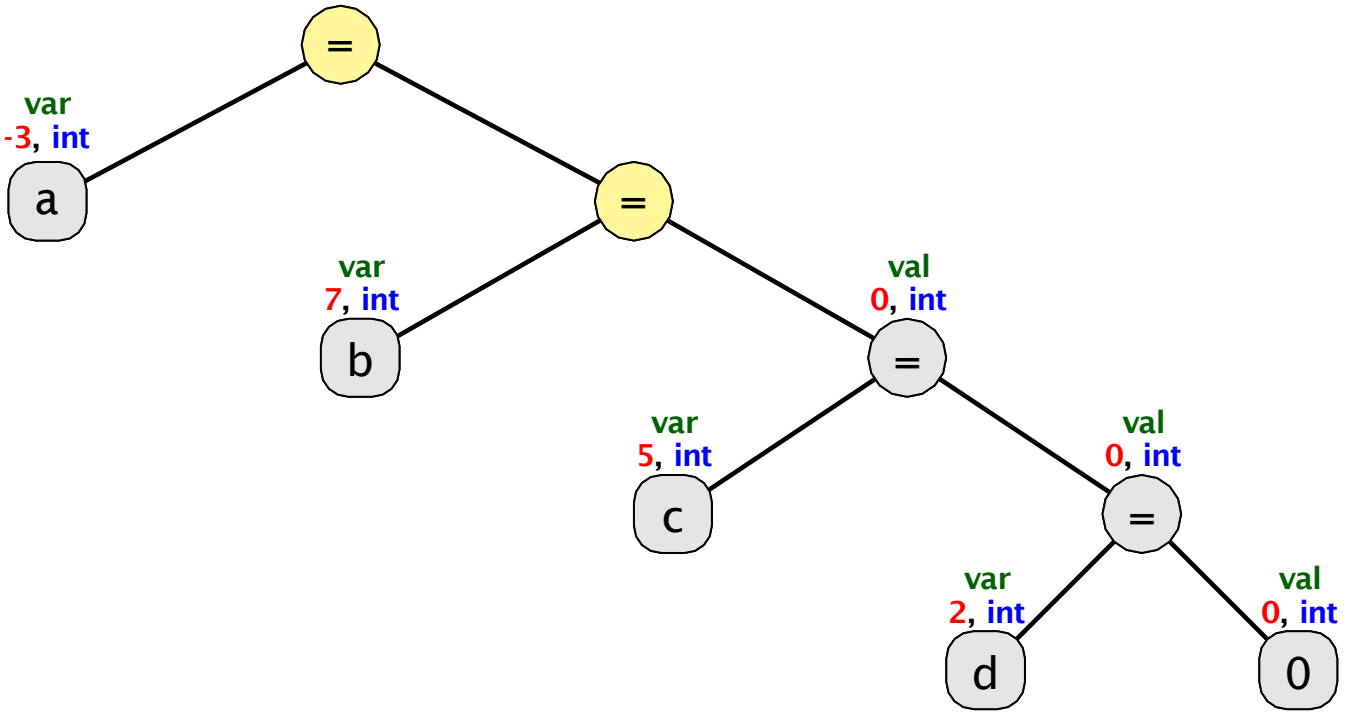
a -3 b 7 c 5 d 0

Beispiel: a = b = c = d = 0



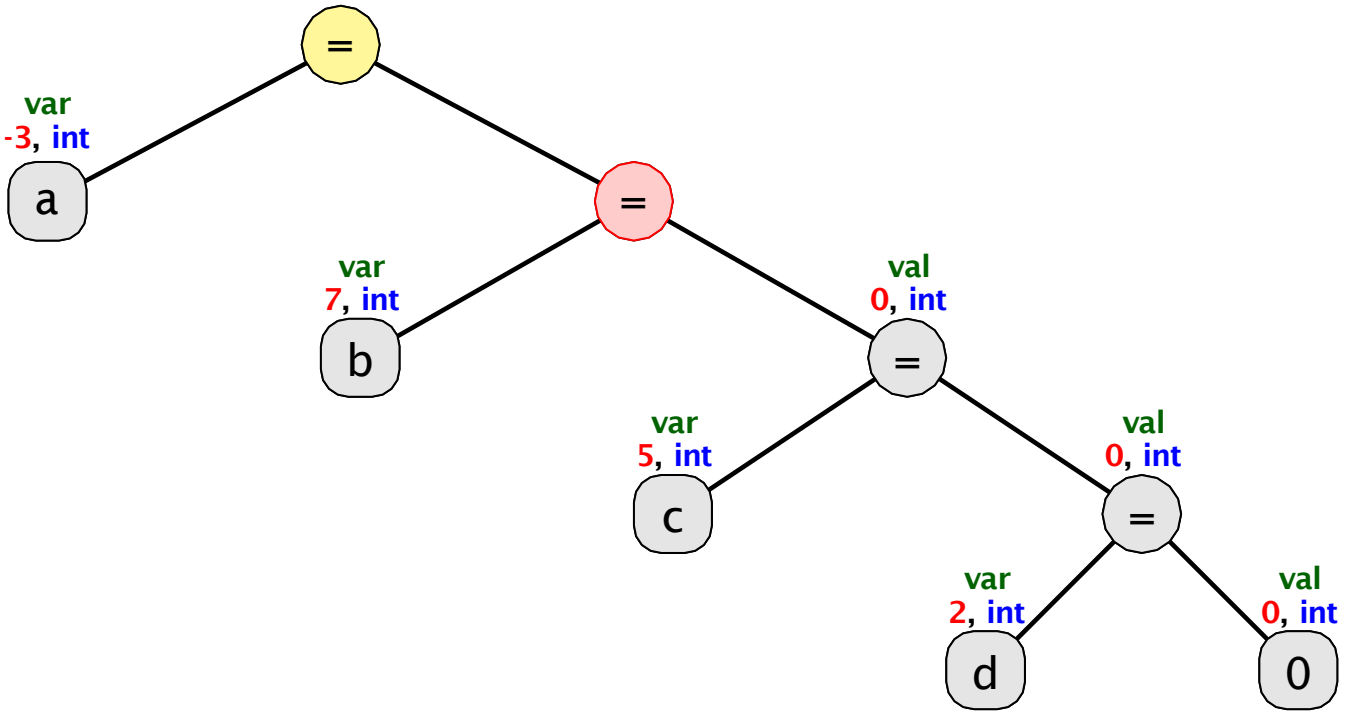
a	b	c	d
-3	7	5	0

Beispiel: a = b = c = d = 0



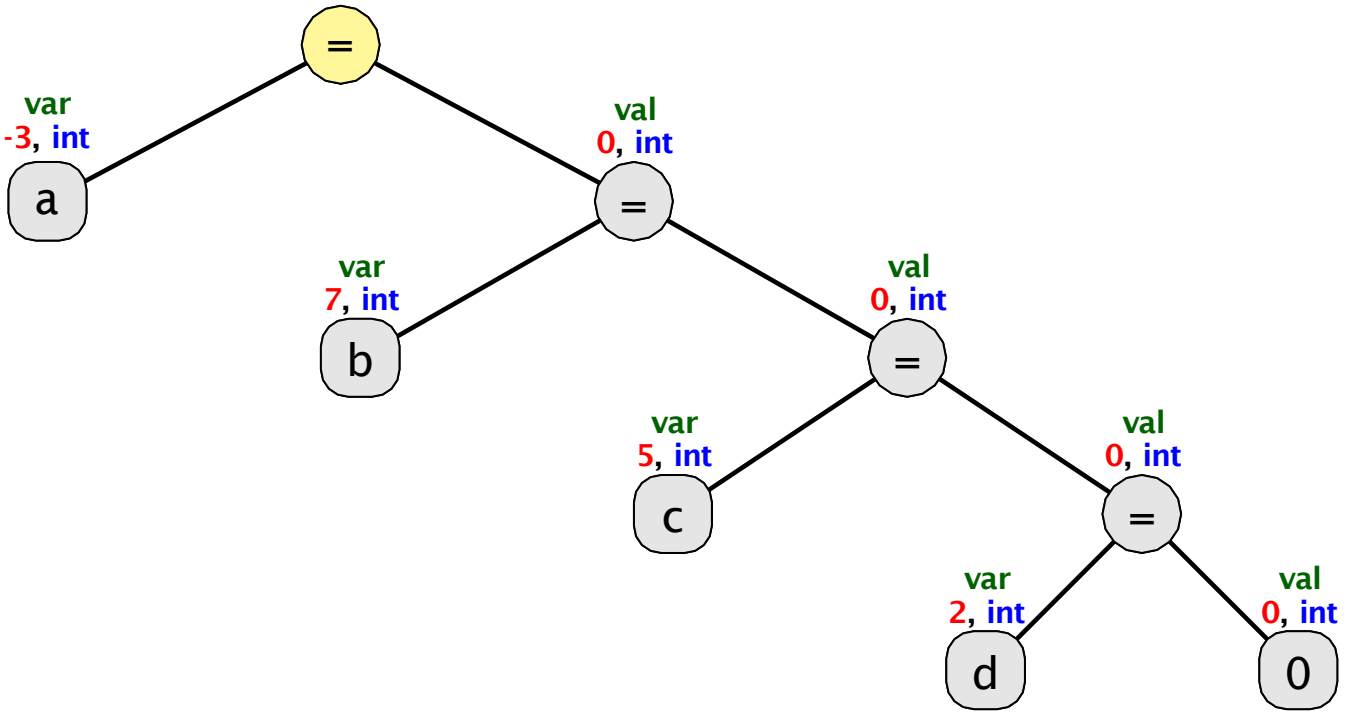
a -3 b 7 c 0 d 0

Beispiel: a = b = c = d = 0



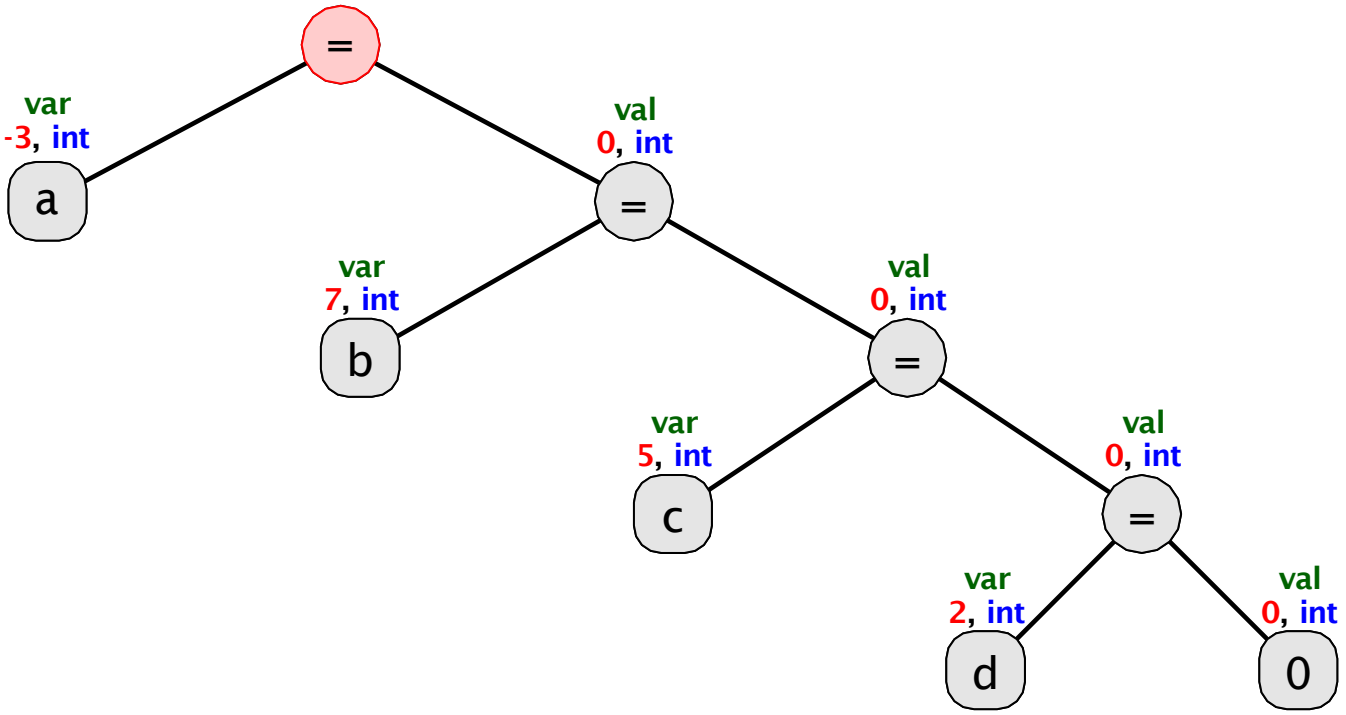
a -3 b 7 c 0 d 0

Beispiel: a = b = c = d = 0



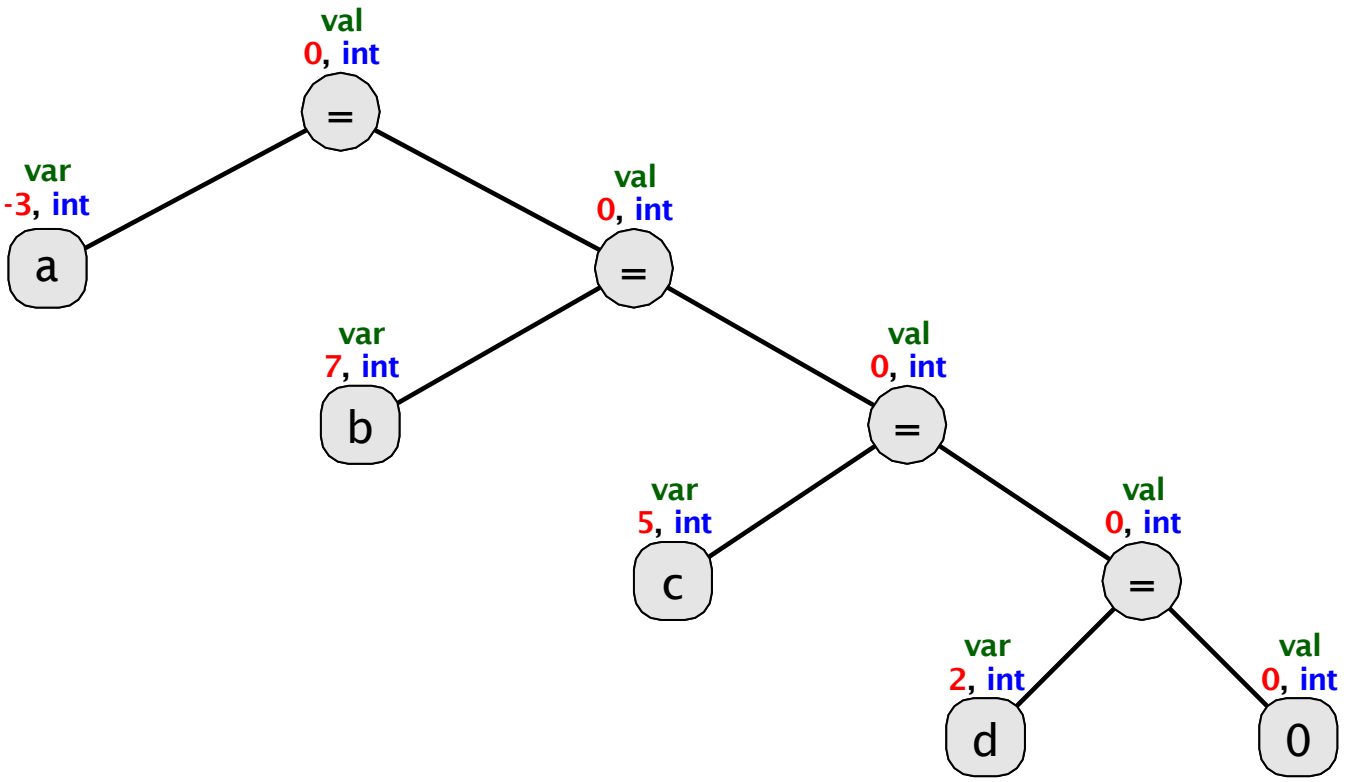
a -3 b 0 c 0 d 0

Beispiel: a = b = c = d = 0



a -3 b 0 c 0 d 0

Beispiel: a = b = c = d = 0

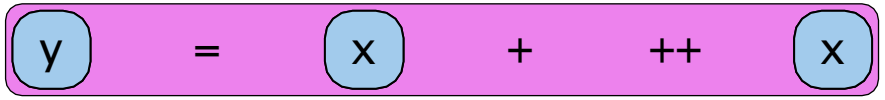


a 0 b 0 c 0 d 0

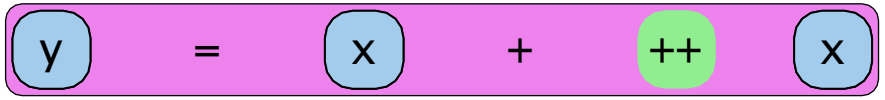
Beispiel: y = x + ++x

y	=	x	+	++	x

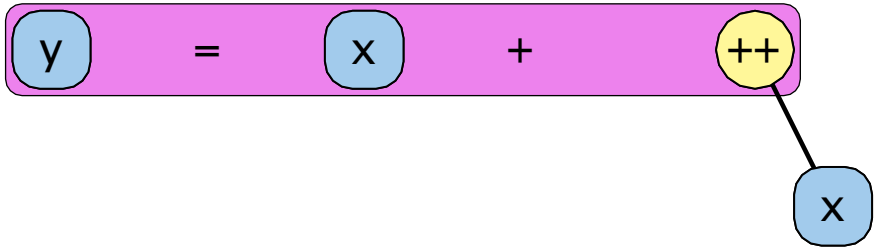
Beispiel: y = x + ++x



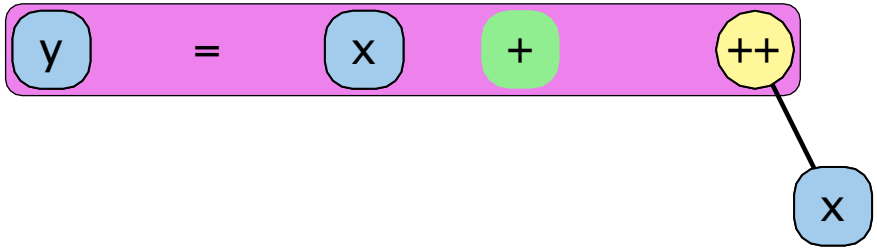
Beispiel: y = x + ++x



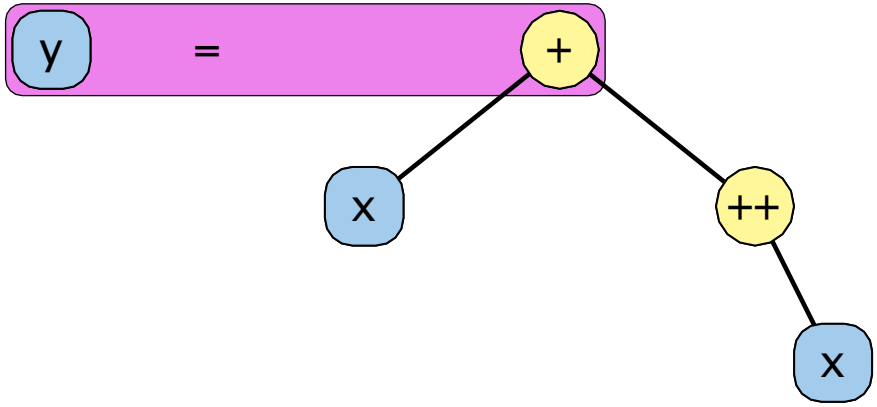
Beispiel: y = x + ++x



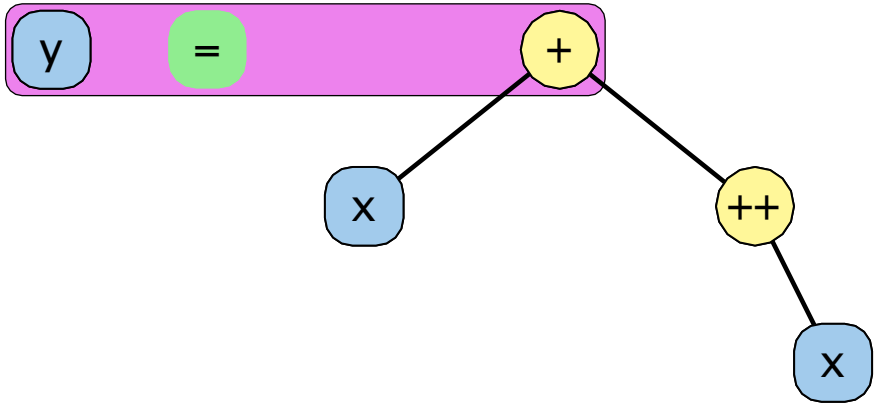
Beispiel: y = x + ++x



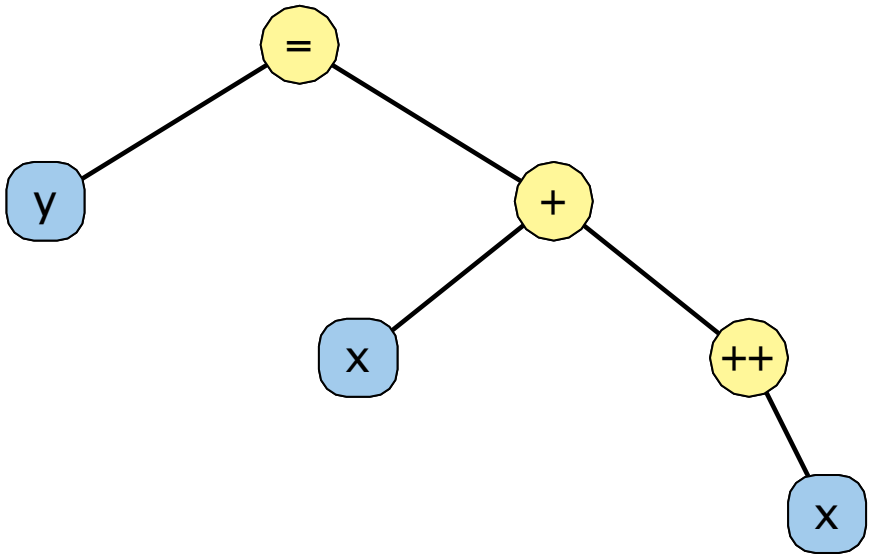
Beispiel: y = x + ++x



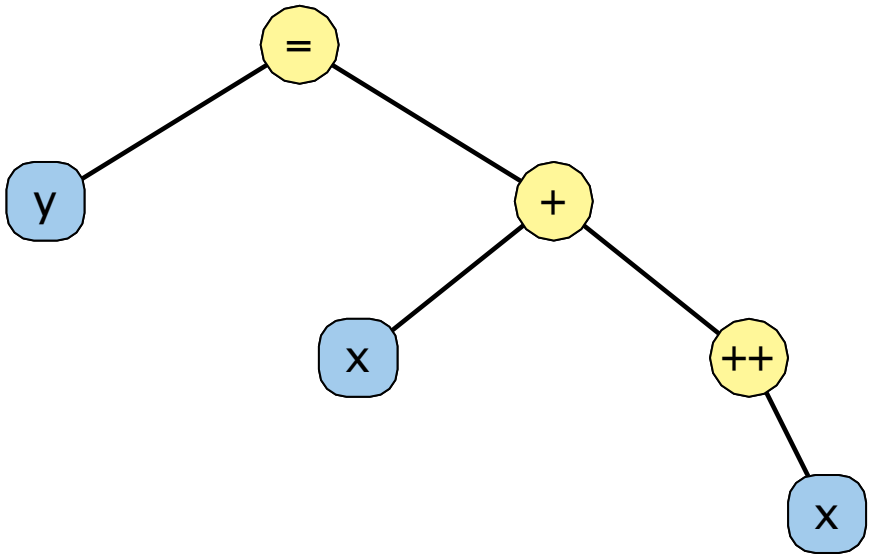
Beispiel: y = x + ++x



Beispiel: y = x + ++x

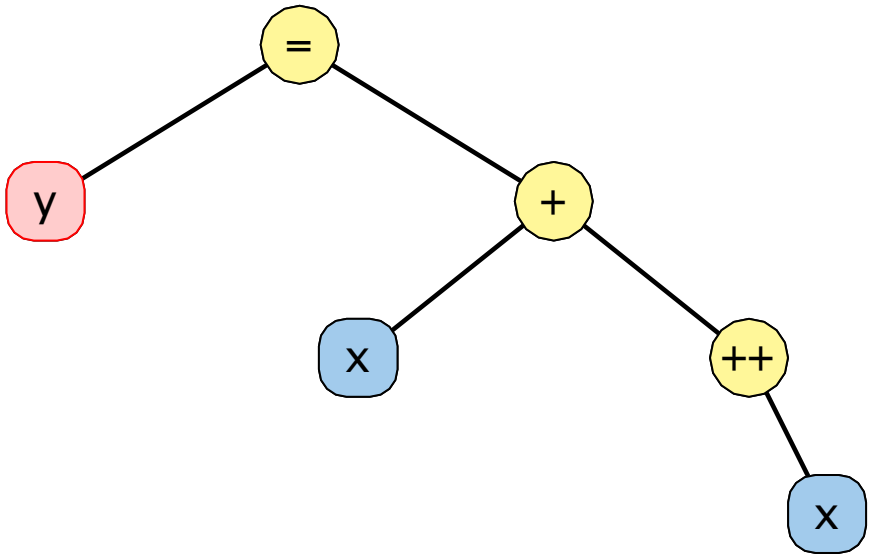


Beispiel: y = x + ++x



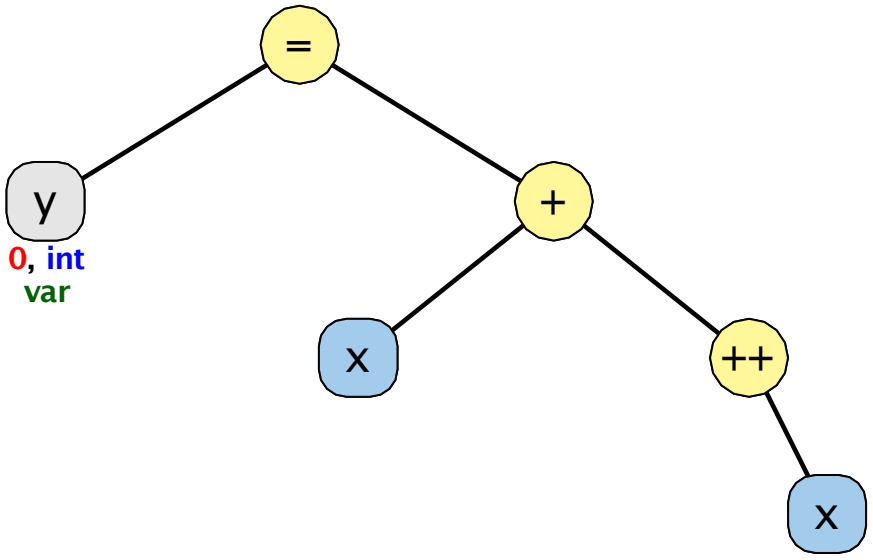
x 4 y 0

Beispiel: y = x + ++x



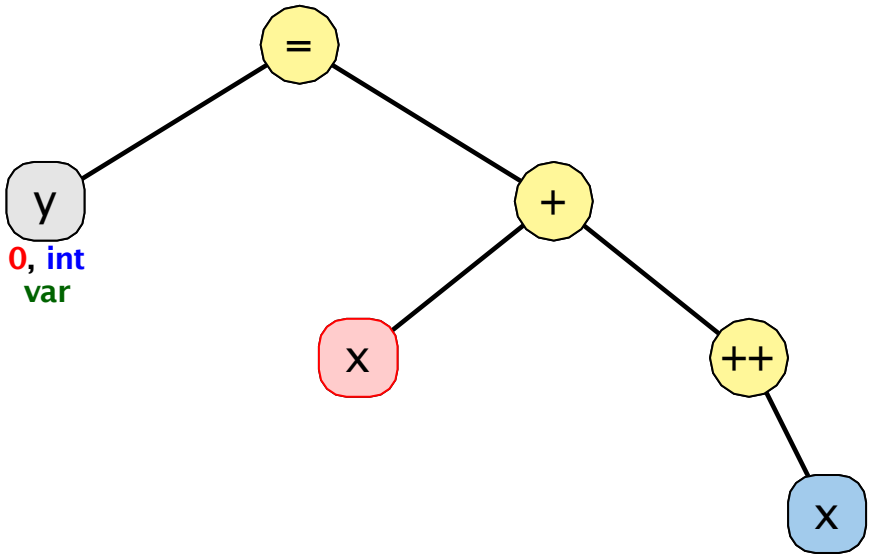
x 4 y 0

Beispiel: y = x + ++x



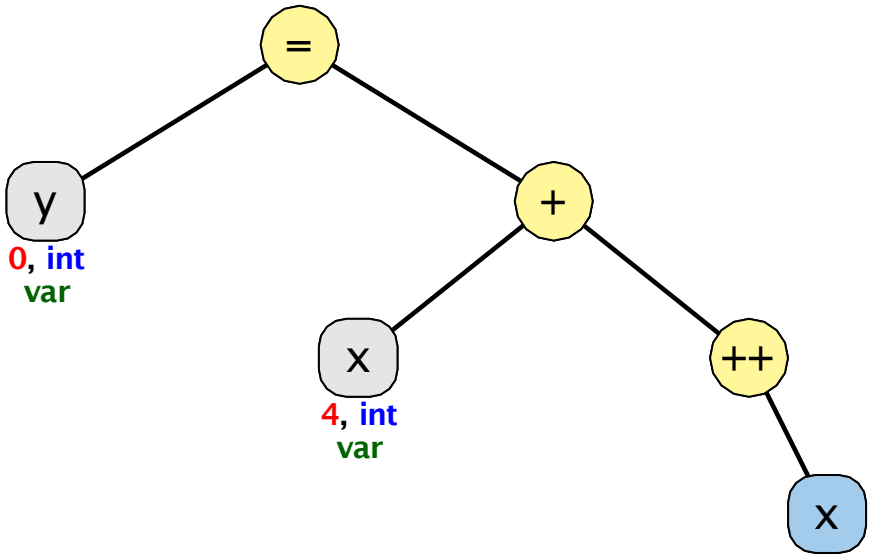
x 4 y 0

Beispiel: y = x + ++x



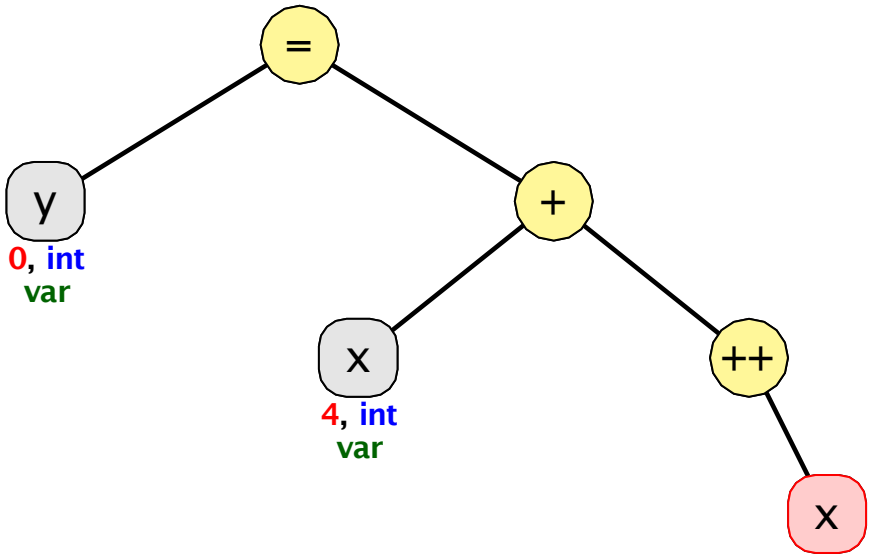
x 4 y 0

Beispiel: y = x + ++x



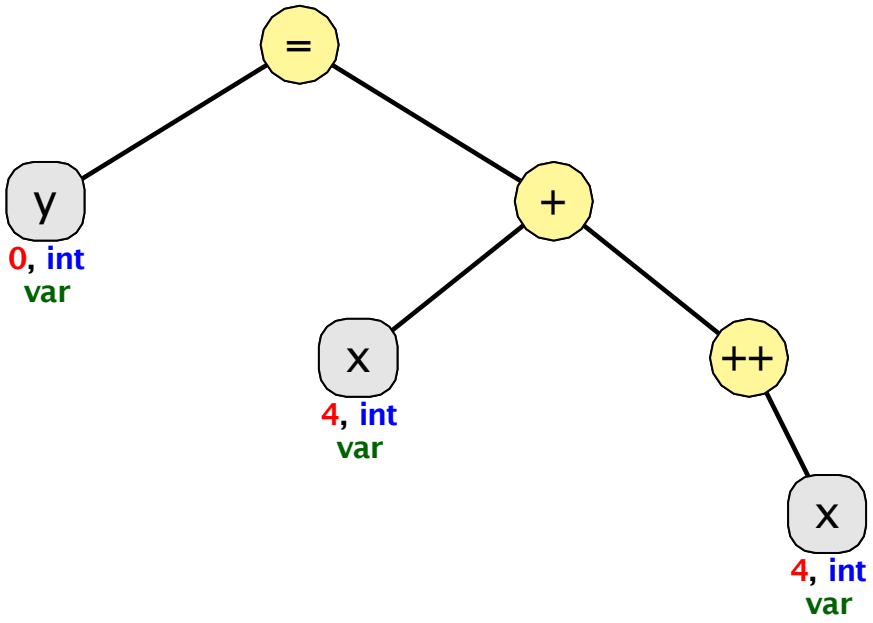
x 4 y 0

Beispiel: y = x + ++x



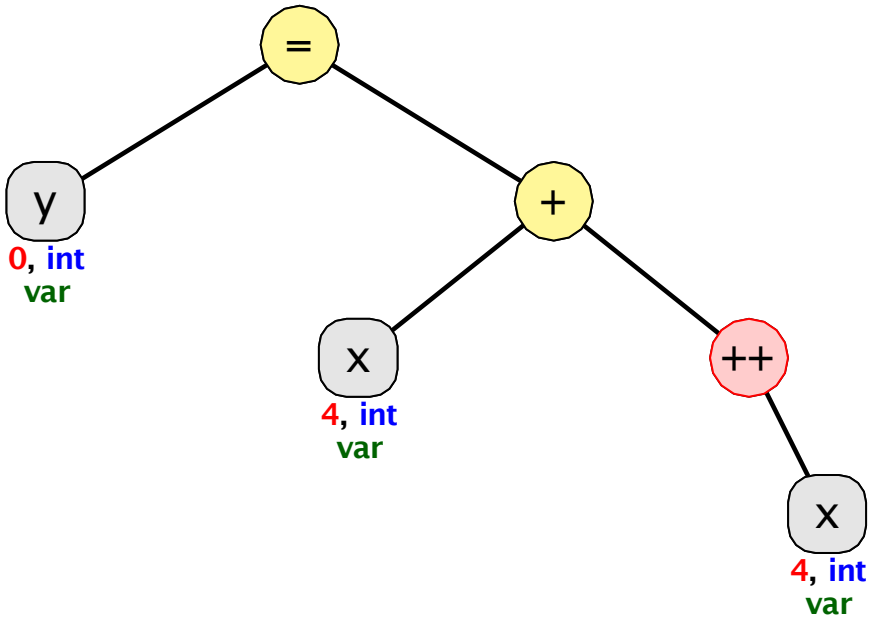
x 4 y 0

Beispiel: y = x + ++x



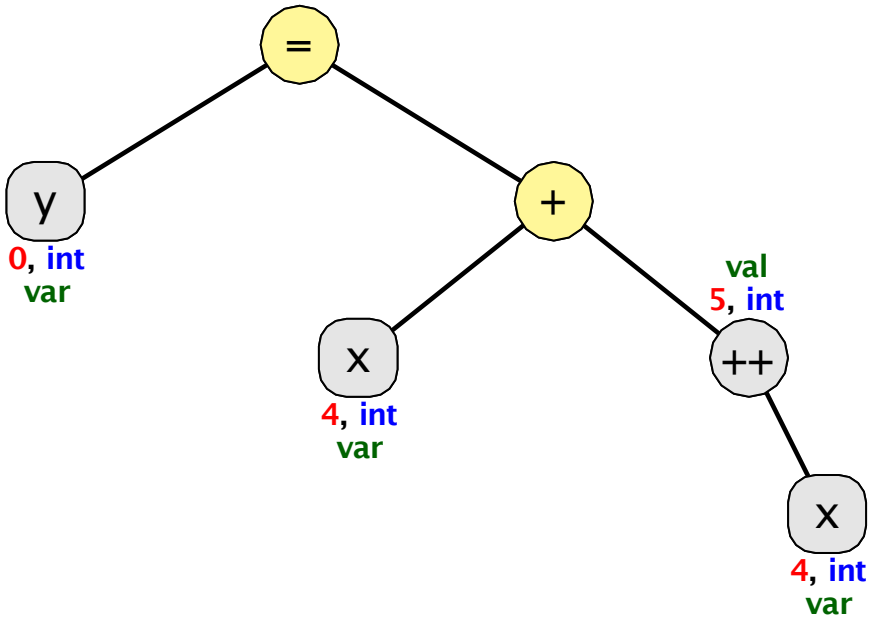
x 4 y 0

Beispiel: y = x + ++x



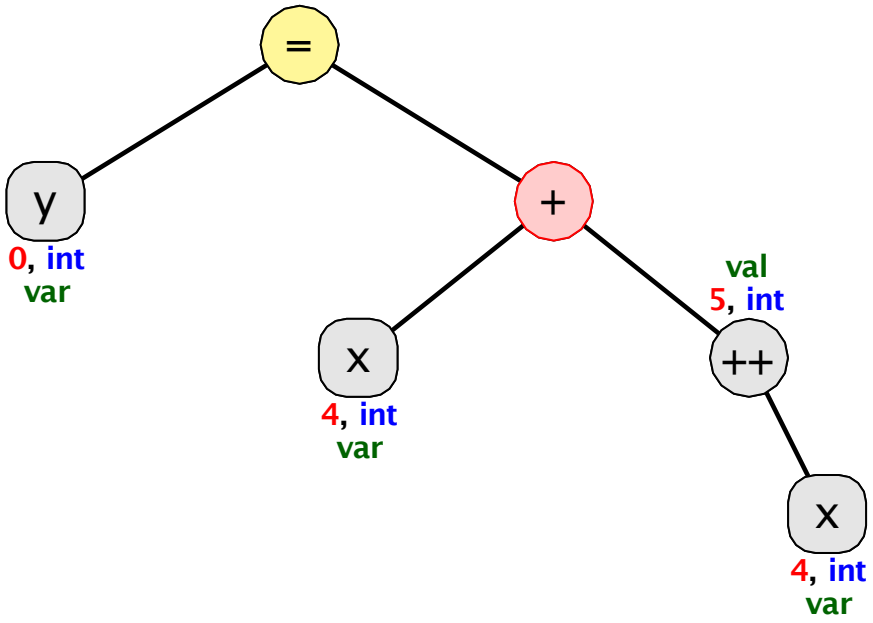
x 4 y 0

Beispiel: y = x + ++x



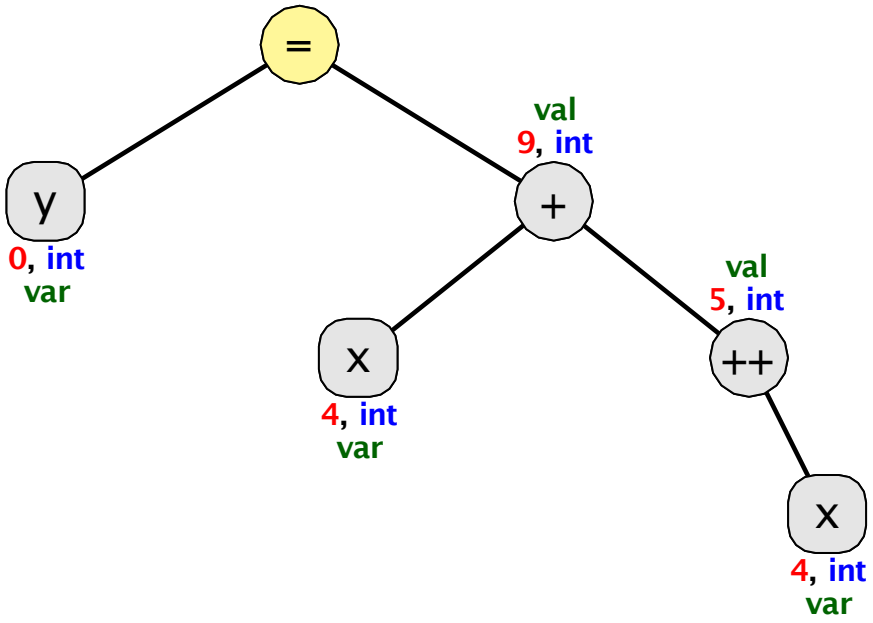
x 5 y 0

Beispiel: y = x + ++x



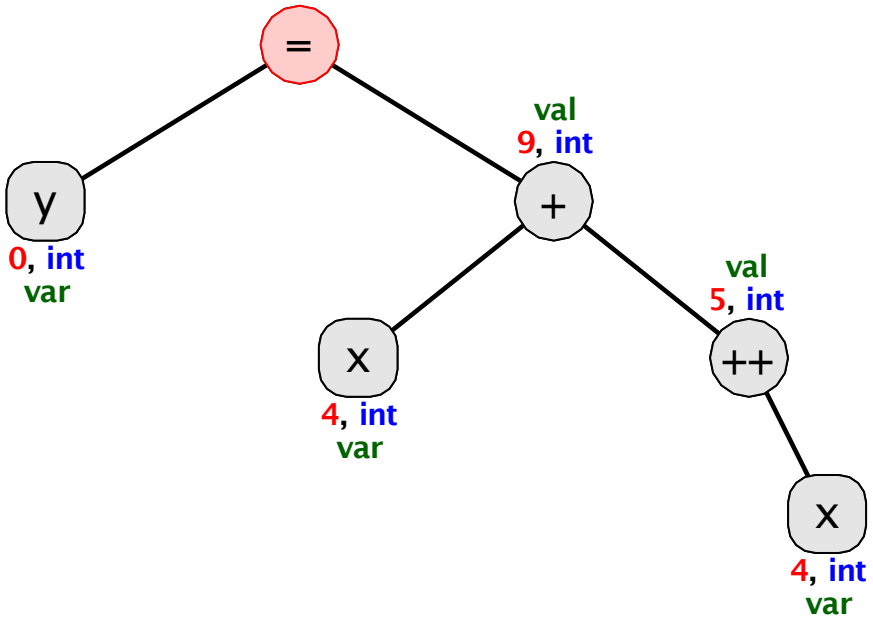
x 5 y 0

Beispiel: y = x + ++x



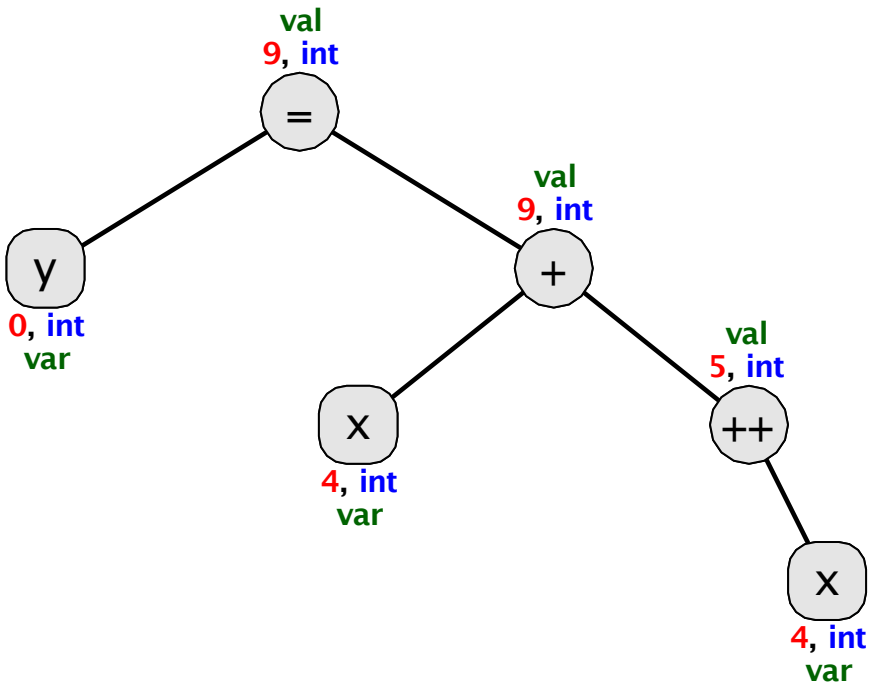
x 5 y 0

Beispiel: y = x + ++x



x 5 y 0

Beispiel: y = x + ++x

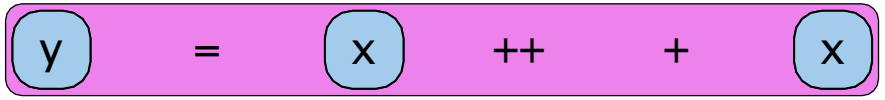


x 5 y 9

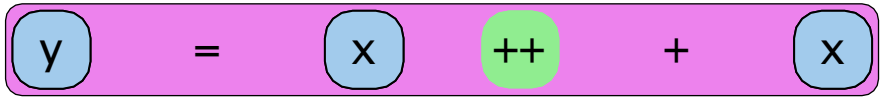
Beispiel: y = x++ + x

y	=	x	++	+	x

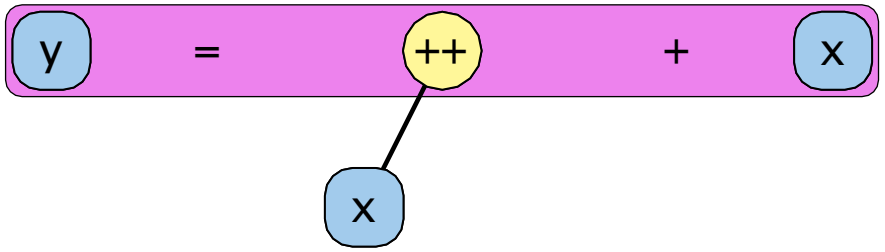
Beispiel: y = x++ + x



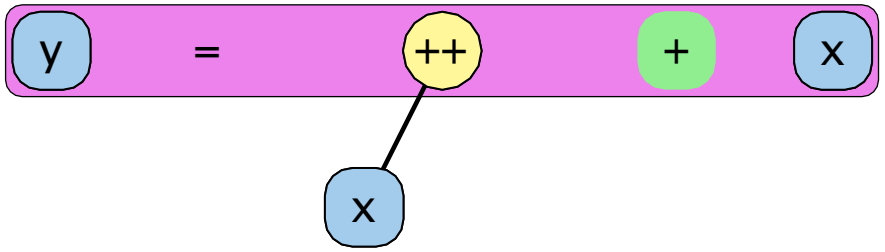
Beispiel: y = x++ + x



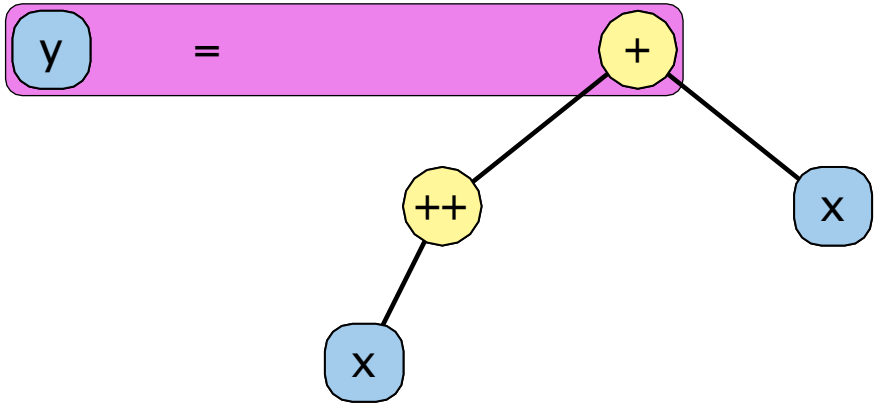
Beispiel: y = x++ + x



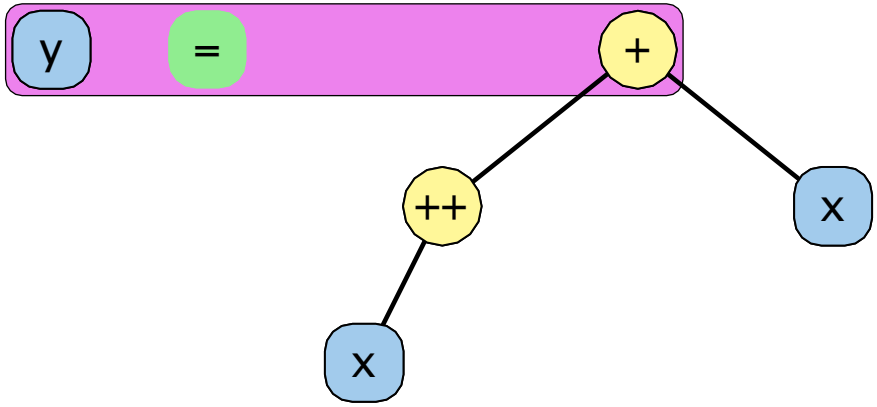
Beispiel: y = x++ + x



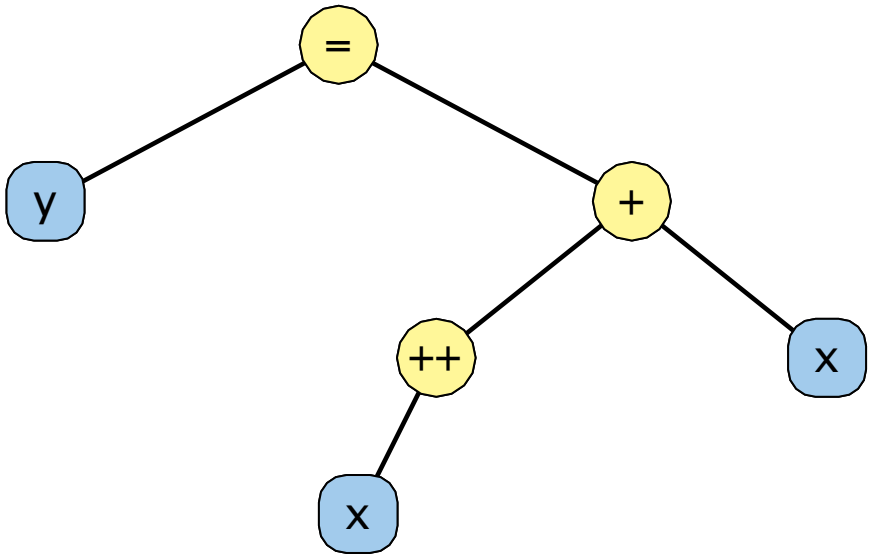
Beispiel: y = x++ + x



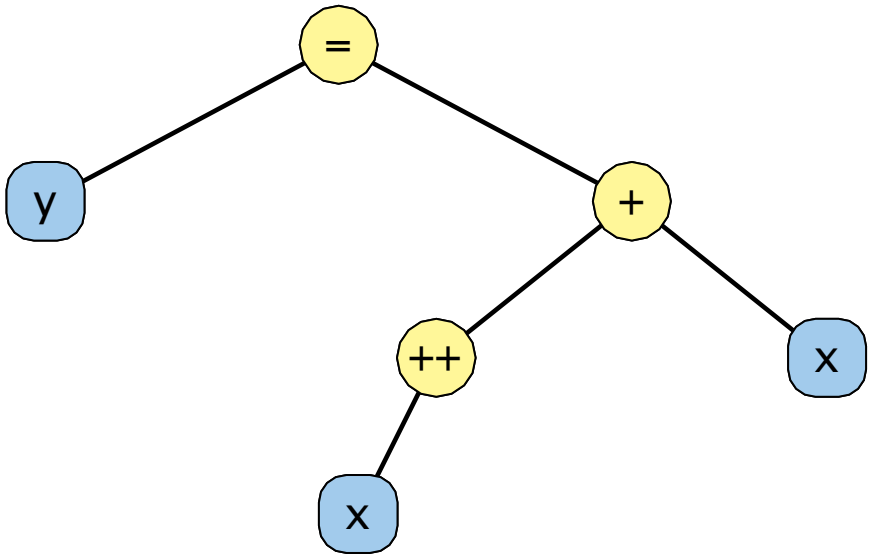
Beispiel: y = x++ + x



Beispiel: y = x++ + x

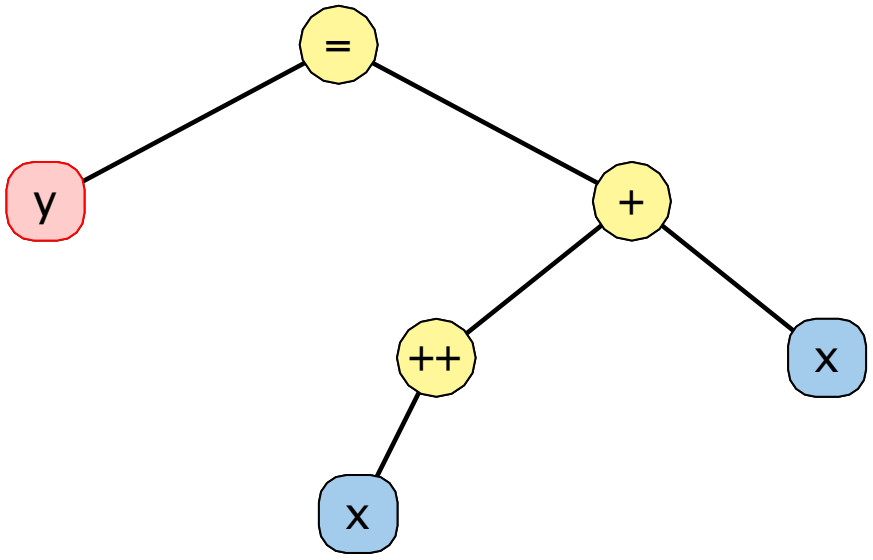


Beispiel: y = x++ + x



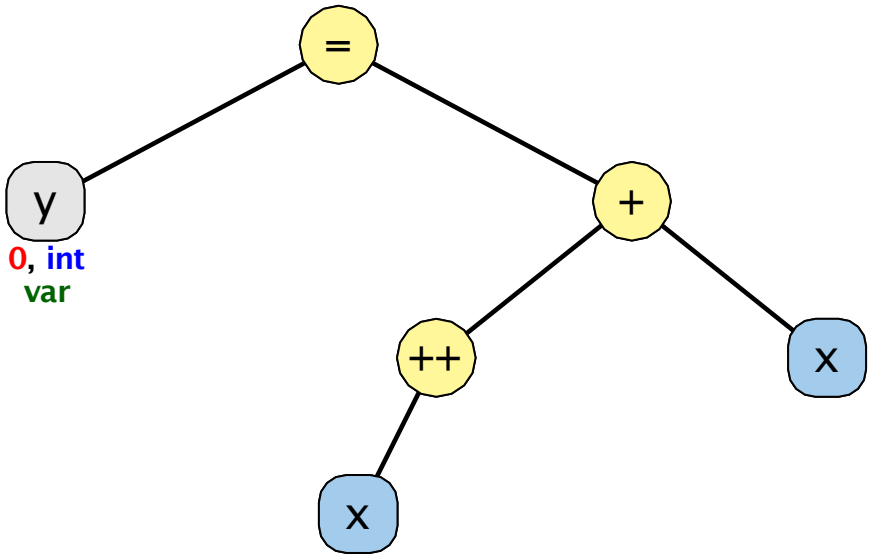
x 4 y 0

Beispiel: y = x++ + x



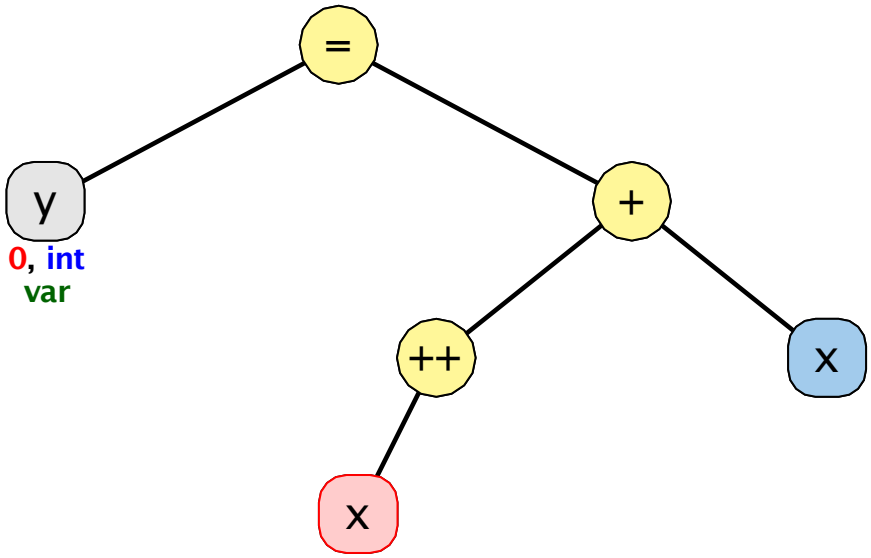
x 4 y 0

Beispiel: y = x++ + x



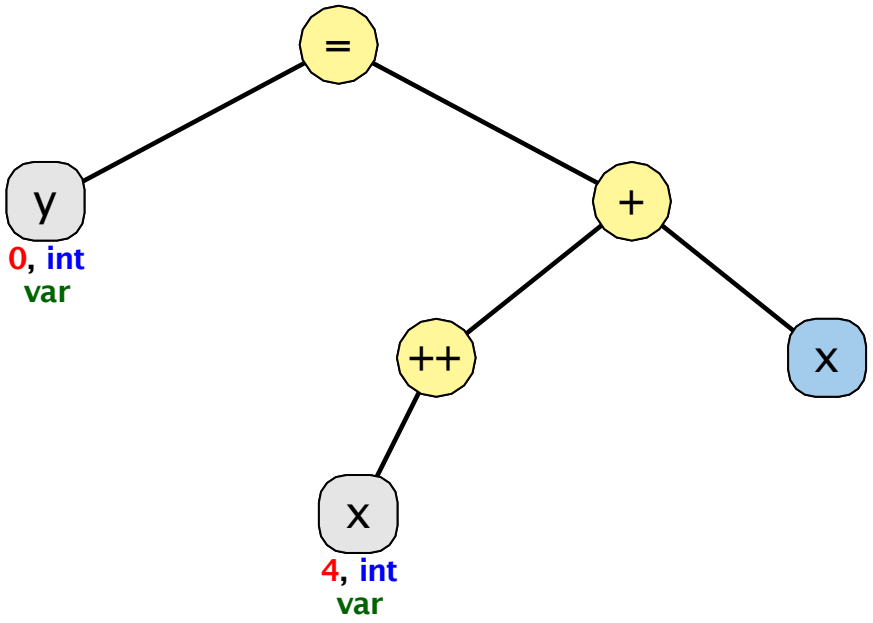
x 4 y 0

Beispiel: y = x++ + x



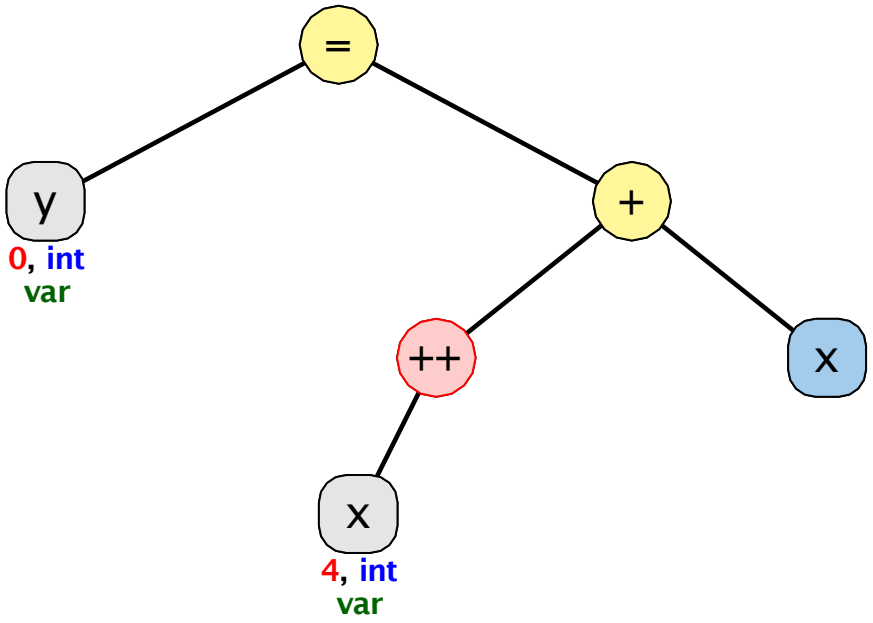
x 4 y 0

Beispiel: y = x++ + x



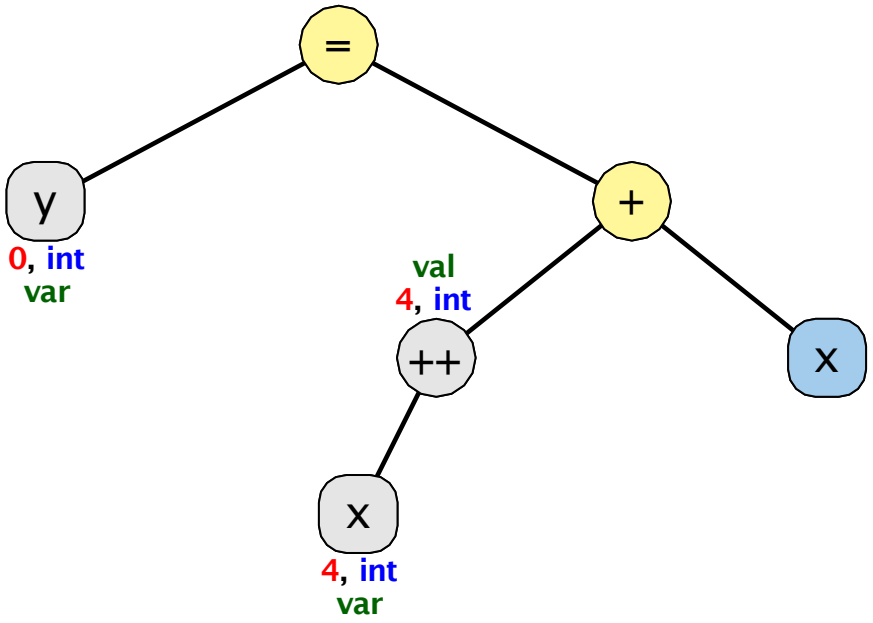
x 4 y 0

Beispiel: y = x++ + x



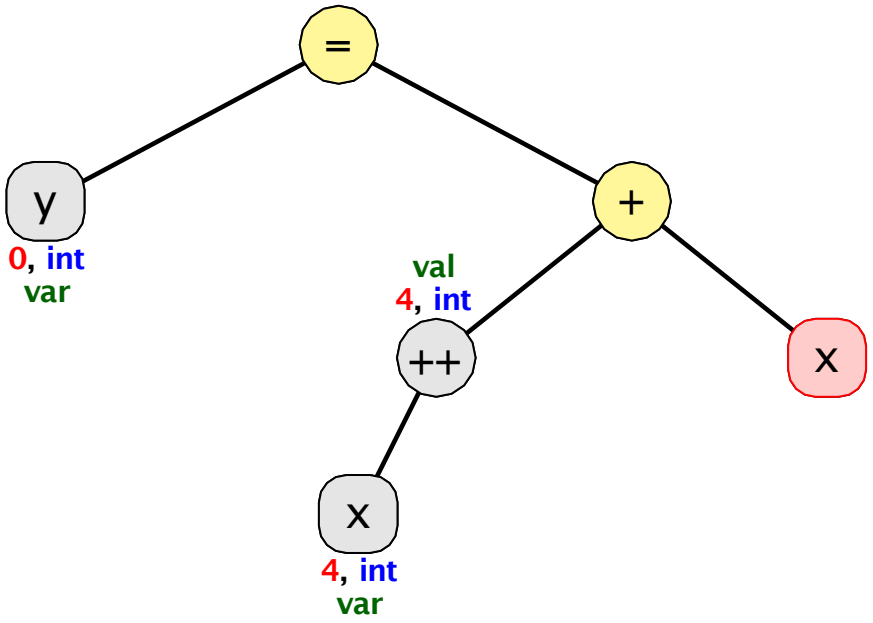
x 4 y 0

Beispiel: y = x++ + x



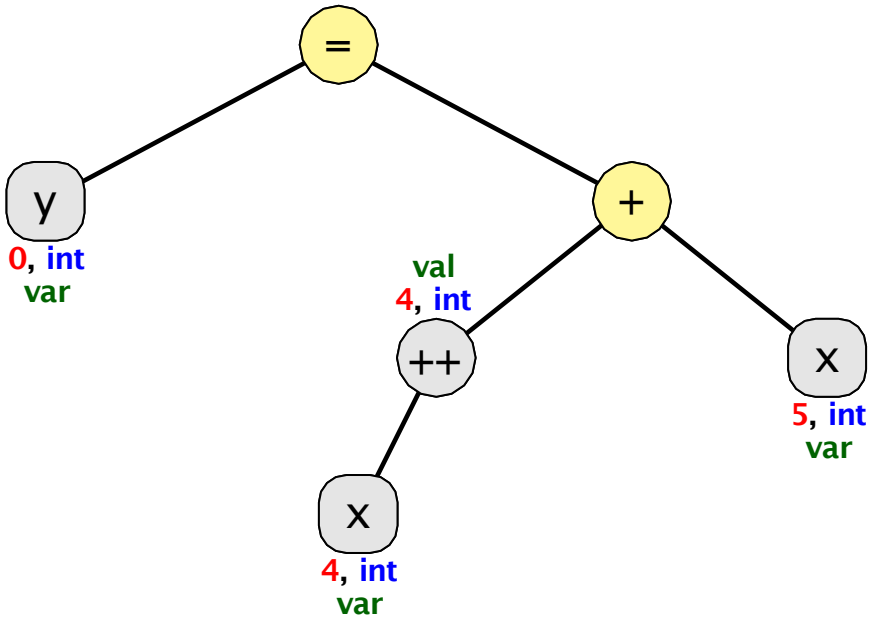
x 5 y 0

Beispiel: y = x++ + x



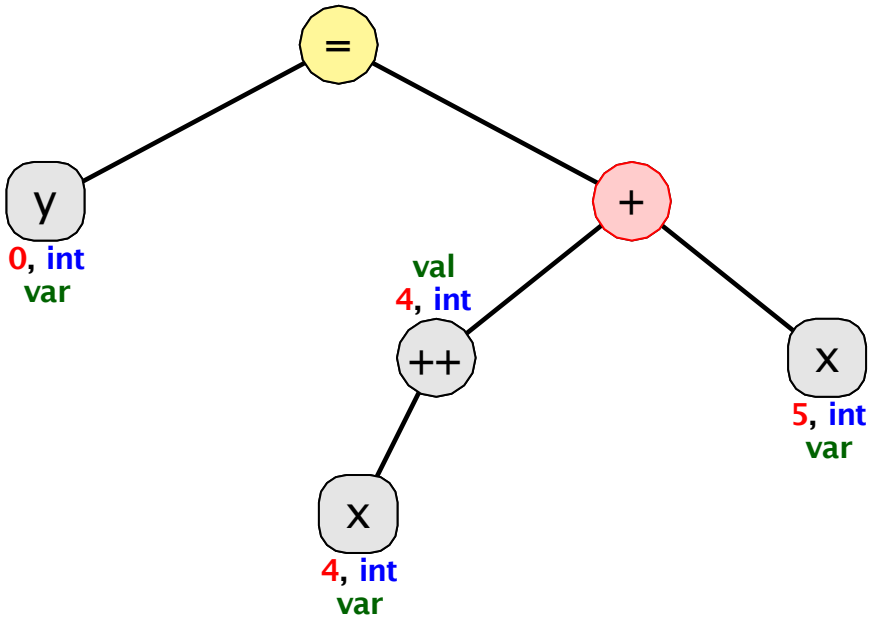
x 5 y 0

Beispiel: y = x++ + x



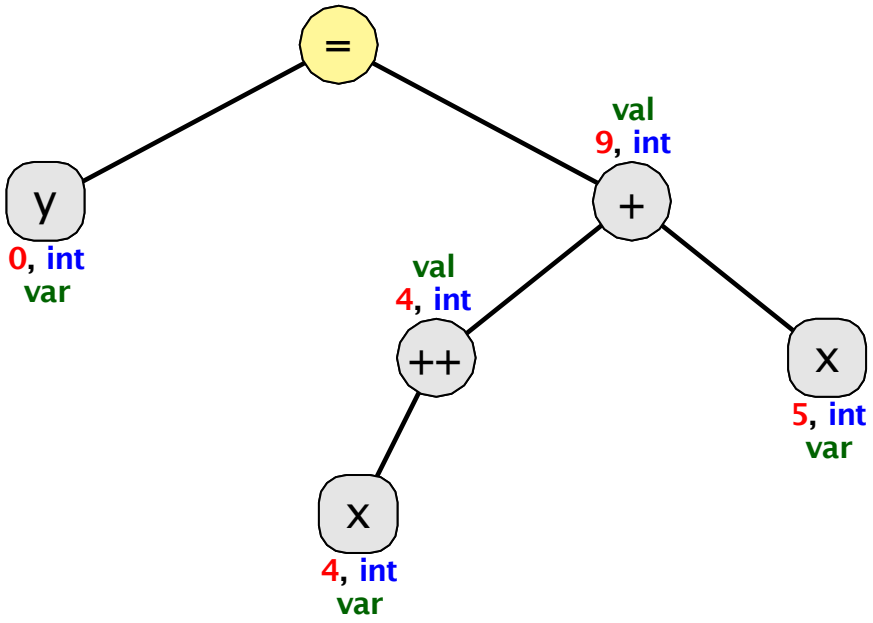
x 5 y 0

Beispiel: y = x++ + x



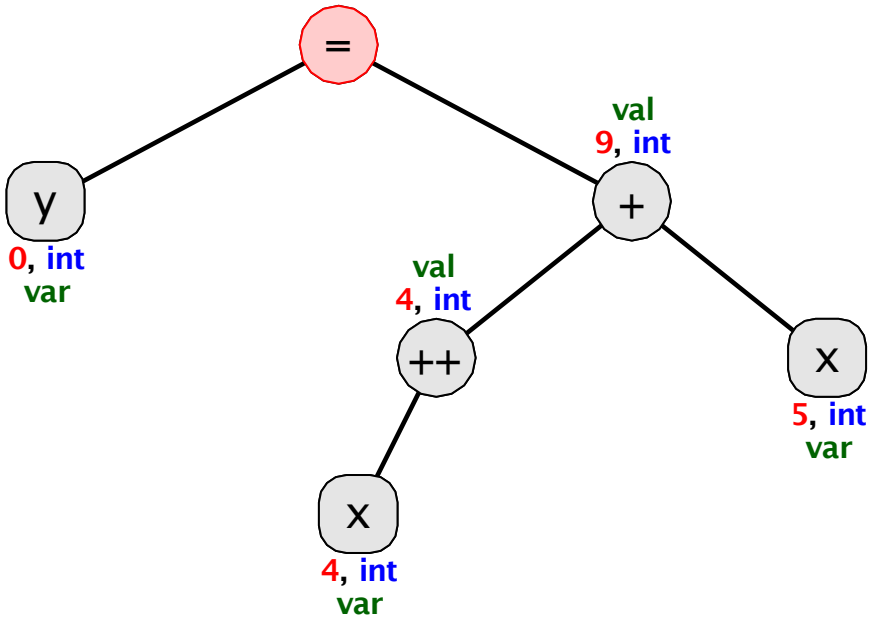
x 5 y 0

Beispiel: y = x++ + x



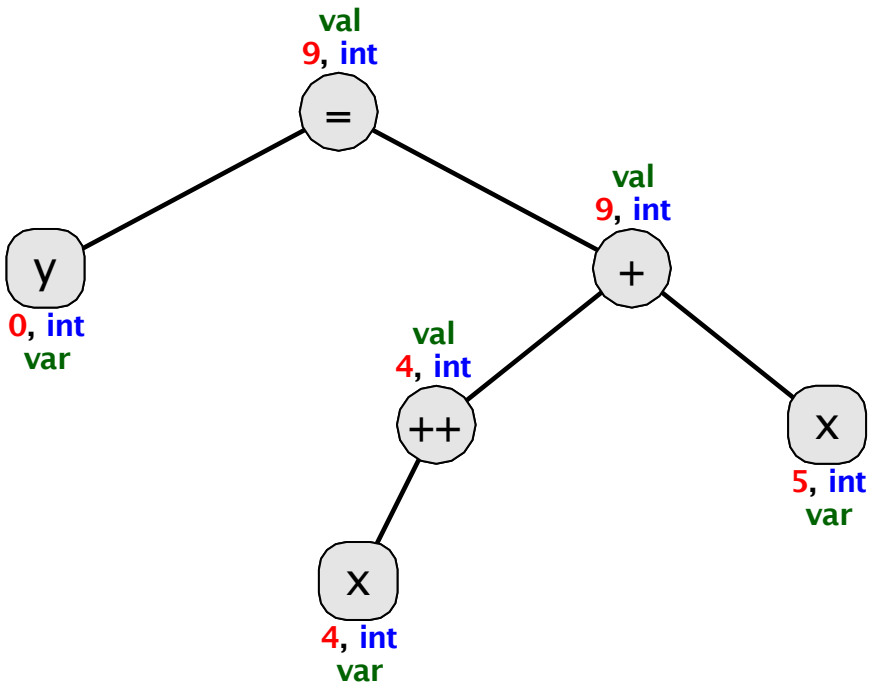
x 5 y 0

Beispiel: y = x++ + x



x 5 y 0

Beispiel: y = x++ + x



x 5 y 9